

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

PROYECTO FIN DE CURSO (PFC) — ENTREGA 4

PROYECTO:

**TIENDATECH: CALIDAD, OBSERVABILIDAD Y
EVALUACIÓN EXPERIMENTAL DEL SISTEMA
DISTRIBUIDO**

MATERIA:

APLICACIONES DISTRIBUIDAS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

CURSO / PERÍODO ACADÉMICO:

7mo A — PERÍODO ACADÉMICO 2026

INTEGRANTES:

Jhinson Stalyn Aucatoma Celorio
Jeremy Ruperto Gaibor Rodriguez
Andy Paul Sanchez Pilaloa
José Alejandro Lozano Morales

FECHA DE ENTREGA:

Martes 25 de agosto del 2026

FECHA PERMITIDO DE ÚLTIMO COMMIT:

Viernes 28 de agosto del 2026 hasta Hora: 23h59

Índice

Resumen	4
Abstract	5
1. Introducción	6
1.1. Problema cuantificado y brecha de requisitos (recuperado y corregido de E1)	6
1.2. Alcance y contribución delimitada de la Entrega 4	7
2. Trabajos relacionados	8
2.1. Bases de datos distribuidas	8
2.2. Arquitectura de microservicios	9
2.3. Ingeniería de aplicaciones móviles	9
2.4. Calidad de software y CI/CD	9
2.5. Matriz comparativa	10
3. Fundamento teórico	12
3.1. Arquitectura en capas y principios SOLID	12
3.2. Patrones de diseño GoF	12
3.3. Pruebas en sistemas distribuidos	13
3.4. Ingeniería de aplicaciones móviles	14
3.5. Integración y entrega continuas (CI/CD)	15
3.5.1. Pipeline CI/CD	15
3.5.2. GitHub Actions y sintaxis YAML	15
3.5.3. Estrategias de despliegue	16
3.5.4. Cobertura de código dentro del pipeline	16
3.6. Observabilidad distribuida	16
3.7. Modelo de calidad ISO/IEC 25010:2023	16

4. Arquitectura del sistema	17
4.1. Vistas C4, ADR de origen y evolución desde E1	17
4.2. Refactor en capas y principios SOLID (D1.1)	18
4.3. Patrones GoF aplicados (D1.2)	19
4.4. Persistencia distribuida operando (D4.2)	20
4.5. Stack tecnológico	21
5. Aplicación web	23
5.1. Funcionalidad (D2.1)	23
5.2. Calidad (D2.2)	24
6. Aplicación móvil	25
6.1. Funcionalidad (D3.1)	25
6.2. Calidad (D3.2)	26
7. Pruebas y CI/CD	27
7.1. Pirámide de pruebas (D5.1)	27
7.2. Contratos de consumidor Pact (D4.1)	31
7.3. Pipeline CI/CD end-to-end (D5.2)	31
8. Observabilidad	37
9. Evaluación ISO/IEC 25010	37
10. Discusión y amenazas a la validez	38
10.1. Interpretación de resultados y alcance no cubierto	38
10.2. Amenazas a la validez (D9.3)	39
10.3. Reflexión ética (D9.2)	39
10.4. Reproducibilidad (D9.4)	40
11. Conclusiones y trabajo futuro	41

11.1. Cierre del ciclo E1–E4	41
11.2. Conclusiones	42
11.3. Trabajo futuro	43

Resumen

Objetivo: cerrar, sobre el sistema distribuido **TiendaTech** validado en la Entrega 3, la brecha entre un sistema que funciona y uno que puede sostenerse, verificarse y evolucionar de forma controlada, mediante refactor arquitectónico, persistencia distribuida segura, calidad de las aplicaciones cliente y automatización del ciclo de integración continua. **Método:** auditoría directa del código fuente frente a cada criterio de la rúbrica de la Entrega 4, declarando explícitamente qué se cumple, qué se cumple de forma parcial y qué queda fuera de alcance, en lugar de reportar cobertura no verificada. **Resultados:** los seis microservicios Java quedaron organizados en cuatro capas con puertos y adaptadores, con cinco patrones GoF documentados con evidencia real; la persistencia distribuida en CockroachDB migró a una conexión cifrada por certificado (mTLS) con *healthchecks* en los siete servicios; la aplicación web expone doce rutas y la aplicación móvil usa dos capacidades del dispositivo con pruebas unitarias e instrumentadas; el pipeline de CI/CD cubre de forma parcial cuatro de los siete *jobs* esperados; y las pruebas del backend, los contratos Pact, la observabilidad distribuida y la evaluación ISO/IEC 25010 quedan con cobertura desigual o explícitamente pendientes. **Conclusiones:** el sistema avanzó de forma genuina pero incompleta hacia la sostenibilidad operativa, documentada con honestidad de alcance en cada sección.

Palabras clave: Microservicios, Arquitectura en capas, Persistencia distribuida, CockroachDB, Calidad de software, CI/CD.

Abstract

Objective: to close, over the **TiendaTech** distributed system validated in Deliverable 3, the gap between a system that works and one that can be sustained, verified, and evolved in a controlled way, through architectural refactoring, secure distributed persistence, client application quality, and continuous integration automation. **Method:** a direct source-code audit against each criterion of the Deliverable 4 rubric, explicitly stating what is fully met, partially met, or out of scope, instead of reporting unverified coverage. **Results:** the six Java microservices were reorganized into four layers with ports and adapters, with five GoF design patterns documented with real code evidence; distributed persistence on CockroachDB migrated to a certificate-based encrypted connection (mTLS) with healthchecks on all seven services; the web application exposes twelve routes and the mobile application uses two device capabilities with unit and instrumented tests; the CI/CD pipeline partially covers four of the seven expected jobs; and backend testing, Pact contracts, distributed observability, and the ISO/IEC 25010 evaluation remain unevenly covered or explicitly pending. **Conclusions:** the system advanced genuinely, though incompletely, toward operational sustainability, documented with honest scope declarations throughout.

Keywords: Microservices, Layered architecture, Distributed persistence, CockroachDB, Software quality, CI/CD.

1. Introducción

1.1. Problema cuantificado y brecha de requisitos (recuperado y corregido de E1)

La Entrega 1 (mayo de 2026) situaba el problema de TiendaTech dentro de un comercio electrónico ecuatoriano cuyas ventas anuales, según cifras de la Cámara Ecuatoriana de Comercio Electrónico (CECE) y la Universidad Espíritu Santo, alcanzarían los USD 4.000 millones¹. Verificar esa cifra para esta entrega final reveló que corresponde específicamente a la proyección de **2022** (un incremento del 24 % frente a los USD 3.220 millones de 2021), no a una cifra vigente de forma indefinida. La proyección más reciente de CECE, para 2025, sitúa la cifra en USD 6.000 millones² — un 50 % más que la cifra que E1 presentó como contexto, y consistente con la proyección independiente de PCMI de USD 6.400 millones para 2024 con una tasa de crecimiento anual compuesta del 20 % hacia 2027³. El fenómeno macroeconómico que motivó el proyecto no solo sigue vigente: se sostiene con una cifra todavía mayor.

La verificación también obliga a corregir, no solo a actualizar, una afirmación de E1: que la categoría de bienes tecnológicos y hardware informático “se posiciona de forma consistente entre los tres sectores con mayor volumen de transacciones”. Ninguna de las fuentes consultadas para esta entrega sostiene ese posicionamiento específico. PCMI reporta tecnología como la quinta categoría por porcentaje de compradores en línea (56 %), detrás de servicios, bienes personales, alimentos/bebidas y entretenimiento; la cobertura de prensa de las proyecciones 2025 de CECE menciona tecnología entre las categorías más compradas, pero sin desglose que la sitúe en el podio por volumen de dólares. Se retira aquí la afirmación de “top tres” en vez de sostenerla sin evidencia — el argumento de fondo del proyecto no depende de que hardware sea la categoría dominante, sino de que la brecha de requisitos formales descrita a continuación es real independientemente del tamaño relativo de la categoría.

Esa brecha, sí, se mantiene sin cambios respecto de E1: el proceso de selección e integración de componentes para el armado de un computador exige validar restricciones de compatibilidad física, eléctrica y lógica entre piezas (por ejemplo, la disparidad de zócalos entre procesador y placa madre). Las plataformas comerciales vigentes en Ecuador cumplen con requisitos transaccionales básicos — gestión de carrito, pasarela de pagos — pero carecen de requisitos lógicos y formales de validación cruzada en tiempo real. Al no existir especificaciones formales automatizadas que validen las restricciones tecnológicas de los productos antes de la compra, el error del consumidor se detecta únicamente tras la entrega física, lo que desencadena ciclos ineficientes de devoluciones y pérdidas económicas. Esta brecha, y no la posición relativa de

¹CECE/UEES, cifra citada en Primicias, <https://www.primicias.ec/noticias/economia/ventas-comercio-electronico-ecuador/>, consultado el 31 de agosto de 2026.

²CECE, cifra citada en El Diario, <https://www.eldiario.ec/negocios/comercio-electronico-en-ecuador-alcanzara-6-000-millones-en-ventas-en-2025-02082025/>, consultado el 31 de agosto de 2026.

³PCMI, *E-Commerce Data Library*, <https://paymentscmi.com/insights/ecuador-datos-mercado-comercio-electronico/>, consultado el 31 de agosto de 2026.

la categoría en el mercado, es la que TiendaTech aborda mediante el motor de compatibilidad descrito en la Sección 4.

La Entrega 3 de **TiendaTech** validó un despliegue contenerizado compuesto por un frontend y seis microservicios conectados a una base de datos distribuida CockroachDB, confirmando el flujo transaccional básico de carrito, orden, factura e inventario sobre esa infraestructura. Esa entrega dejó explícitamente fuera de su contribución delimitada el aislamiento de persistencia por servicio, la alta disponibilidad productiva, el motor de compatibilidad de componentes y el asistente conversacional, así como la calidad de código, la observabilidad y la evaluación experimental formal del sistema.

La Entrega 4 retoma el sistema en ese punto y se concentra en cerrar la brecha entre *tener un sistema distribuido que funciona* y *tener un sistema distribuido que puede sostenerse, verificarse y evolucionar de forma controlada*. En este período se completó la migración de los microservicios afectados hacia una organización en capas explícita (**presentation/application/domain/infrastructure**), se documentaron los patrones de diseño GoF aplicados, se aseguró la persistencia distribuida con una conexión cifrada por certificados (mTLS) hacia el clúster CockroachDB en lugar de la conexión directa sin cifrar heredada de E3, y se avanzó en la calidad de las aplicaciones cliente (web y móvil) y en la automatización del ciclo de integración continua.

1.2. Alcance y contribución delimitada de la Entrega 4

Siguiendo el mismo principio de honestidad de alcance aplicado en las entregas anteriores, esta sección declara explícitamente qué criterios de la Guía Integral de la Entrega 4 cubre este documento y cuáles quedan fuera.

Se abordan de forma directa: el refactor en capas y los patrones GoF con su respectivo registro de decisión arquitectónica (D1); el estado de las aplicaciones web y móvil, tanto en funcionalidad como en calidad de código (D2, D3); la persistencia distribuida operando de forma segura y, en la medida en que el tiempo del equipo lo permitió, los contratos de consumidor entre los clientes y el backend (D4); la pirámide de pruebas y el pipeline de integración continua (D5); la trazabilidad de la evolución del proyecto entre E1 y E4 (D8.3); y la reflexión ética, las amenazas a la validez y la reproducibilidad del entorno de despliegue (D9).

Quedan explícitamente fuera del alcance de esta entrega la instrumentación de observabilidad distribuida (OpenTelemetry, Prometheus, Grafana) y, como consecuencia directa de ello, la evaluación experimental basada en ISO/IEC 25010 (D6 y D7), dado que esta última depende de contar con las métricas operativas que solo la primera puede producir. Las Secciones 8, 9 y 10 retoman esta decisión de alcance con más detalle. Esta exclusión se declara aquí de forma explícita, antes de que el lector avance en el documento, para no generar la expectativa de una cobertura que el equipo no pudo verificar en el tiempo disponible.

2. Trabajos relacionados

Esta sección revisa quince fuentes indexadas, todas con DOI verificado — dos ya citadas en `docs/entrega3/PFC3.tex` y reutilizadas aquí por seguir siendo pertinentes, cinco incorporadas para la Entrega 4, y ocho recuperadas del acervo bibliográfico acumulativo del equipo (`docs/entrega3/referenciasPFC.bib`), donde ya existían verificadas pero sin citarse todavía en esta sección — organizadas en los cuatro ejes que exige la rúbrica de la Entrega 4: bases de datos distribuidas, arquitectura de microservicios, ingeniería de aplicaciones móviles y calidad de software/CI-CD.

2.1. Bases de datos distribuidas

Taft, Sharif, Matei et al. [1] describen la arquitectura de CockroachDB — el motor efectivamente usado por TiendaTech desde E3 — detallando cómo fragmenta las tablas en rangos replicados por consenso Raft y las lecciones aprendidas al adaptar Raft a un sistema de producción (agrupar *heartbeats*, pausar grupos Raft inactivos, reconfiguración por consenso conjunto). Özsu y Valduriez [2] complementan esto con el marco formal de fragmentación horizontal primaria y derivada que TiendaTech ya usó para justificar el particionamiento de `pedidos.orden` por trimestre en E3 (ADR-003). Ninguna de las dos fuentes evalúa, sin embargo, el costo operativo de migrar una conexión insegura a una conexión mTLS con certificados cliente sobre un clúster ya desplegado — que es exactamente el cambio de infraestructura documentado en la Sección 4.4 de esta entrega.

Kang, Jiang y Blanas [3] proponen un motor de almacenamiento para fragmentación *irregular* de tablas — particiones que no siguen un patrón uniforme de rangos ni de hash — que describe con precisión el problema que ADR-003 resolvió de forma manual: CockroachDB Community Edition rechaza `PARTITION BY RANGE` por ser una capacidad Enterprise, así que TiendaTech materializa los límites trimestrales mediante `SPLIT AT` y `SCATTER` en vez de una partición declarativa nativa. Zhou, Li, Feng et al. [4] atacan el mismo problema con una alternativa basada en aprendizaje de grafos, la contracara automatizada del enfoque manual y basado en reglas que TiendaTech adoptó por restricción de la versión comunitaria del motor, no por preferencia de diseño. Liu, Li y Chen [5] sitúan ambos enfoques dentro de un panorama más amplio de técnicas de particionamiento (rango, hash, por grafo, híbridas), útil para justificar por qué TiendaTech eligió rango trimestral y no una alternativa más sofisticada: el volumen de datos del proyecto académico no justifica la complejidad operativa de un particionador aprendido. Abdelhafiz [6] ofrecen la panorámica arquitectónica general de bases de datos distribuidas (fragmentación, replicación, transparencia de ubicación) que sirve de marco introductorio a las tres fuentes anteriores. Hasan y Yasmin [7] documentan las anomalías y soluciones de control de concurrencia en bases de datos distribuidas — directamente relevante para TiendaTech, que verificó de forma empírica una colisión bajo aislamiento `SERIALIZABLE` controlada intencionalmente (`docs/evidencias/colision-serializable-controlada.md`), en vez de solo argumentar en abstracto que el nivel de aislamiento es correcto. Ninguna de las cinco fuentes evalúa un caso con las restricciones concretas de TiendaTech: una versión

comunitaria sin particionamiento declarativo, corriendo sobre un clúster de solo tres nodos con fines académicos.

2.2. Arquitectura de microservicios

Márquez, Osses y Astudillo [8] revisan sistemáticamente la literatura académica e industrial sobre patrones y tácticas arquitectónicas para microservicios, identificando 44 patrones en fuentes académicas y 74 en fuentes industriales, la mayoría asociados a escalabilidad, flexibilidad, capacidad de prueba, desempeño y elasticidad. Zhong, Li, Huang et al. [9] investigan empíricamente — combinando una revisión sistemática de 34 estudios con una encuesta a 63 profesionales — el uso de *Domain-Driven Design* para identificar los límites (*bounded contexts*) de cada microservicio, un problema de diseño previo al que TiendaTech resolvió de forma pragmática y no formalizada al dividir el dominio en usuarios, productos, inventario, pedidos, órdenes a proveedores y ventas. Ninguna de las dos fuentes profundiza en el refactor interno de un microservicio ya existente hacia una arquitectura de puertos y adaptadores en cuatro capas — que es precisamente lo que la Sección 4 documenta para los seis servicios Java de TiendaTech.

2.3. Ingeniería de aplicaciones móviles

Nawrocki, Wrona, Marczak et al. [10] comparan empíricamente aplicaciones idénticas desarrolladas con frameworks nativos y multiplataforma (Kotlin nativo frente a React Native y Flutter, entre otros), evaluando funcionalidad, esfuerzo de desarrollo y rendimiento; sus resultados muestran que el desarrollo nativo ofrece el mayor rendimiento y el acceso más directo a las capacidades del dispositivo, a costa de mayor esfuerzo de desarrollo por no compartir código entre plataformas. Este hallazgo es directamente relevante para TiendaTech: la aplicación móvil (Sección 6) se construyó en Kotlin nativo con Jetpack Compose precisamente para acceder sin capas intermedias a CameraX/ML Kit (escaneo de códigos de barras) y a la API de notificaciones del sistema operativo, las dos capacidades de dispositivo requeridas por la rúbrica — una elección que este trabajo relacionado ayuda a justificar cuantitativamente, aunque TiendaTech mismo no ejecutó un benchmark comparativo propio frente a una alternativa multiplataforma.

2.4. Calidad de software y CI/CD

Shahin, Babar y Zhu [11] ofrecen la revisión sistemática de referencia sobre integración, entrega y despliegue continuos, clasificando enfoques, herramientas, desafíos y prácticas de la industria; su marco conceptual de *pipeline* como *quality gate* automatizado es el que se usa para evaluar, en la Sección 7.3, qué tan completo está el pipeline real de TiendaTech frente al ideal de siete *jobs* de la rúbrica. Hui, Wang, Li et al. [12] aportan el estudio más reciente y más específico para este proyecto: un mapeo sistemático de métodos de prueba

para sistemas basados en microservicios, que identifica nueve categorías de pruebas (incluidas pruebas unitarias, de contrato y extremo a extremo) y los desafíos de aplicarlas cuando los servicios se despliegan y actualizan de forma independiente — exactamente la situación de TiendaTech, donde la cobertura de pruebas es desigual entre sus seis microservicios (Sección 7.1).

Decan, Mens, Mazrae et al. [13] estudian empíricamente cómo se usa GitHub Actions en repositorios reales, encontrando que la mayoría de los *workflows* definen pocos *jobs* y los reconfiguran con poca frecuencia una vez estabilizados — un patrón que TiendaTech no sigue: el equipo mantiene dos *workflows* paralelos (`ci.yml` y `ci-cd.yml`, Sección 7.3) precisamente porque el segundo siguió evolucionando activamente. Mazrae, Mens, Golzadeh et al. [14] analizan cualitativamente por qué los equipos migran o combinan herramientas de CI/CD, un marco que ayuda a nombrar con precisión la deuda técnica que TiendaTech ya declara explícitamente: no ha completado la migración de `ci.yml` a `ci-cd.yml`, y sigue pagando el costo de mantener ambos. Saroar y Nayebi [15] encuestan a desarrolladores sobre su percepción de GitHub Actions, reportando que la depuración de fallos de *workflow* y la comprensión de la sintaxis YAML son las mayores fuentes de fricción — consistente con los dos incidentes reales que el equipo tuvo que depurar en esta entrega: la propiedad inexistente `jacoco.check.skip` (en vez de la correcta `jacoco.skip`) y el contexto de *build* de Docker mal configurado para los servicios que dependen de `contracts/` fuera de su propio directorio (Sección 7.3). Ninguna de las cinco fuentes de este eje mide el costo de retrasar la adopción de pruebas de contrato (Pact) cuando el tiempo del equipo es limitado, que es la decisión de priorización que TiendaTech documenta explícitamente en la Sección 7.2.

2.5. Matriz comparativa

La Tabla 1 condensa el aporte de cada fuente y la brecha que deja abierta respecto del trabajo realizado en TiendaTech.

Cuadro 1: Matriz comparativa de trabajos relacionados (Entrega 4).

Referencia	Aporte principal	Brecha frente a TiendaTech E4
Taft, Sharif, Matei et al. [1]	Arquitectura de CockroachDB y adaptación de Raft a producción.	No evalúa migrar una conexión insegura a mTLS sobre un clúster ya desplegado.
Özsu y Valduriez [2]	Marco formal de fragmentación horizontal primaria/derivada.	Supone conocida la carga de consultas desde el diseño inicial.
Kang, Jiang y Blanas [3]	Motor de almacenamiento para fragmentación irregular de tablas.	No cubre el caso de una edición comunitaria sin particionamiento declarativo nativo.
Zhou, Li, Feng et al. [4]	Sistema de particionamiento de bases de datos basado en aprendizaje de grafos.	Requiere infraestructura de entrenamiento que un clúster académico de tres nodos no justifica.
Liu, Li y Chen [5]	Panorama comparativo de técnicas de particionamiento (rango, hash, grafo, híbridas).	No evalúa el costo operativo de simular rango declarativo con SPLIT AT/SCATTER.
Abdelhafiz [6]	Panorámica arquitectónica de bases de datos distribuidas.	No aborda casos con restricciones de licenciamiento de la edición comunitaria.
Hasan y Yasmin [7]	Anomalías y soluciones de control de concurrencia distribuida.	No incluye una prueba empírica de colisión bajo SERIALIZABLE con reintento medido.
Márquez, Osses y Astudillo [8]	44+74 patrones/tácticas arquitectónicas para microservicios.	No cubre el refactor interno de un servicio ya existente hacia capas.
Zhong, Li, Huang et al. [9]	DDD para identificar límites de microservicios (34 estudios, 63 encuestados).	No aborda la reestructuración posterior de un microservicio ya delimitado.
Nawrocki, Wrona, Marczak et al. [10]	Comparación empírica nativo vs. multiplataforma (funcionalidad, esfuerzo, rendimiento).	TiendaTech no ejecutó un benchmark propio frente a una alternativa multiplataforma.
Shahin, Babar y Zhu [11]	Revisión sistemática de CI/CD: enfoques, herramientas, prácticas.	No mide el costo de un pipeline parcialmente implementado bajo restricción de tiempo.
Hui, Wang, Li et al. [12]	Mapeo sistemático de pruebas para sistemas basados en microservicios.	No mide el costo de retrasar la adopción de pruebas de contrato (Pact).
Decan, Mens, Mazrae et al. [13]	Estudio empírico del uso real de GitHub Actions en repositorios.	No modela el caso de dos <i>workflows</i> paralelos durante una migración inconclusa.
Mazrae, Mens, Golzadeh et al. [14]	Análisis cualitativo de migración y uso de herramientas de CI/CD.	No cuantifica el costo de mantener dos <i>workflows</i> redundantes activos simultáneamente.
Saroar y Nayebi [15]	Encuesta sobre la percepción de desarrolladores acerca de GitHub Actions.	No cubre los incidentes concretos de sintaxis/configuración que TiendaTech depuró en E4.

3. Fundamento teórico

3.1. Arquitectura en capas y principios SOLID

La arquitectura en capas separa un sistema en niveles con responsabilidades distintas y dependencias unidireccionales: presentación (adapta el mundo exterior al sistema), aplicación (orquesta casos de uso), dominio (reglas de negocio y contratos) e infraestructura (detalles técnicos concretos: bases de datos, HTTP, colas). En un sistema distribuido este patrón de *N capas* se aplica dos veces: dentro de cada microservicio, para ordenar su propio código, y entre microservicios, donde la capa de infraestructura de un servicio suele ser, en realidad, la interfaz de red hacia otro servicio. Esta segunda aplicación exige más disciplina que la primera: si el dominio de un servicio depende directamente del cliente HTTP concreto de otro servicio, cualquier cambio de protocolo (de REST a gRPC, de síncrono a mensajería asíncrona) se propaga hasta la lógica de negocio en vez de quedar contenido en un adaptador.

Los principios SOLID — responsabilidad única (SRP), abierto/cerrado (OCP), sustitución de Liskov (LSP), segregación de interfaces (ISP) e inversión de dependencias (DIP) — son las restricciones que evitan ese acoplamiento. Aplicados a microservicios adquieren un matiz adicional frente a su lectura clásica dentro de un monolito: el SRP no solo separa responsabilidades dentro de una clase, sino que delimita qué lógica de negocio pertenece a un servicio y cuál a otro; y el DIP es el principio que hace posible sustituir un colaborador remoto — por un doble de prueba, o por una implementación alternativa del mismo contrato — sin modificar la capa de aplicación, siempre que esta dependa de una interfaz (un *puerto*) y no de un cliente concreto. Este acoplamiento reducido se materializa en la práctica mediante *inyección de dependencias*: un contenedor de inversión de control (IoC) — Spring en el ecosistema Java, u otros equivalentes en otros *frameworks* — resuelve en tiempo de arranque qué implementación concreta satisface cada interfaz, de modo que ninguna clase de negocio instancia directamente a sus colaboradores [16]. El uso disciplinado de SOLID es, en particular, el prerrequisito para que los patrones de diseño GoF aporten valor real: un patrón aplicado sobre un diseño que ya viola SOLID solo añade complejidad accidental.

3.2. Patrones de diseño GoF

El catálogo de patrones de diseño de Gamma, Helm, Johnson y Vlissides — conocidos como el "Gang of Four" (GoF) — documenta veintitrés soluciones recurrentes a problemas de diseño orientado a objetos, agrupadas en patrones creacionales, estructurales y de comportamiento [17]. Su valor no está en aplicarlos por cumplir una cuota, sino en que cada patrón resuelve un problema de acoplamiento o variabilidad concreto.

En sistemas distribuidos, la literatura sobre microservicios señala un subconjunto recurrente de estos patrones — junto con *Repository*, atribuido a Fowler y no al catálogo GoF original, pero de adopción extendida en este contexto — como los de mayor uso práctico [18]:

- **Repository** (Fowler): abstrae el acceso a los datos de un servicio detrás de una interfaz de dominio, de modo que la lógica de negocio no conoce si la persistencia ocurre en una base de datos relacional, documental o en memoria; es el patrón más común para satisfacer el DIP entre **application** e **infrastructure** descrito en la subsección anterior.
- **Factory Method** (creacional): encapsula la construcción de un objeto — típicamente un cliente remoto o una conexión — detrás de un método que las subclasses o la configuración pueden sobrescribir, evitando que el código que lo consume conozca su clase concreta ni sus parámetros de construcción.
- **Proxy** (estructural): representa localmente un objeto remoto o costoso de crear; en microservicios es el patrón detrás de los *stubs* HTTP/gRPC generados y de envoltorios que añaden *circuit breaking* o caché sin que quien lo invoca note la diferencia frente a llamar al objeto real.
- **Observer** (comportamiento): desacopla a un productor de eventos de sus consumidores, que se suscriben sin que el productor conozca su identidad ni su cantidad; es la base conceptual de la mensajería asíncrona y del patrón *publish/subscribe* entre servicios.
- **Strategy** (comportamiento): encapsula un algoritmo o una decisión de negocio intercambiable (por ejemplo, distintos proveedores de pago, notificación o explicación) detrás de una interfaz común, permitiendo elegir la implementación en tiempo de ejecución en vez de dispersar condicionales por el código.
- **Decorator** (estructural): añade comportamiento — registro, reintentos, cabeceras de autenticación — a un objeto existente envolviéndolo, en vez de modificar su clase o crear una jerarquía de subclasses para cada combinación posible de comportamientos.

Los seis comparten el mismo objetivo: introducir un punto de indirección (una interfaz) donde antes había una dependencia concreta, de modo que un cambio en el colaborador — de protocolo, de proveedor, de algoritmo — no obligue a modificar a quien lo consume. La auditoría de cuáles de estos patrones están genuinamente presentes en el código de TiendaTech — y cuáles se evaluaron y se descartaron por no tener estructura GoF real — se documenta con evidencia concreta en la Sección 4, subsección "Patrones GoF aplicados (D1.2)", en vez de asumirse aquí.

3.3. Pruebas en sistemas distribuidos

En un sistema basado en microservicios no es conveniente depender de un solo tipo de prueba. Cada servicio puede funcionar correctamente por separado y, aun así, presentar fallos cuando se comunica con los demás. Por esta razón, la estrategia de pruebas se organiza como una pirámide formada por pruebas unitarias, de integración, de contrato y de extremo a extremo. La mayor cantidad corresponde a las pruebas unitarias, porque son rápidas y permiten revisar reglas de negocio o funciones aisladas. Las pruebas de integración comprueban el trabajo

conjunto con elementos como la base de datos, el API Gateway u otros servicios. En los niveles superiores se encuentran las pruebas de contrato y las de extremo a extremo, que abarcan interacciones más completas, pero también requieren más tiempo y recursos para ejecutarse [16], [19].

Las pruebas de contrato son especialmente útiles cuando un servicio consume la API de otro. Un cambio aparentemente pequeño, como modificar el nombre de un atributo o el formato de una respuesta, puede afectar al consumidor aunque el proveedor continúe funcionando. Pact aplica el enfoque *Consumer-Driven Contract Testing*, en el cual el consumidor define las respuestas que espera y el proveedor verifica que puede cumplirlas. De esta manera, se pueden detectar incompatibilidades antes de desplegar una nueva versión del servicio [16].

Otro aspecto importante es conocer cómo responde el sistema cuando varias personas lo utilizan al mismo tiempo. Herramientas como Locust o k6 permiten simular usuarios concurrentes y registrar datos como el número de solicitudes atendidas, el tiempo de respuesta y la cantidad de errores. Estas pruebas ayudan a observar si el rendimiento se degrada al aumentar la carga y a identificar cuál servicio puede convertirse en un cuello de botella.

La cobertura de código sirve como apoyo para conocer qué partes del programa fueron ejecutadas durante las pruebas. Un porcentaje alto permite detectar menos zonas sin verificar, pero no garantiza por sí solo que el sistema esté libre de errores, ya que también importa la calidad de los casos diseñados. Por eso la cobertura debe analizarse junto con los resultados de las pruebas y no únicamente como una cifra que se debe alcanzar [19].

Finalmente, las pruebas pueden complementarse con herramientas de análisis estático. SonarCloud permite revisar aspectos como duplicación, complejidad y posibles problemas de mantenibilidad; Pylint se utiliza en proyectos Python y ESLint en aplicaciones JavaScript o TypeScript. Estas herramientas examinan el código sin ejecutar el sistema y permiten detectar problemas que podrían pasar desapercibidos durante una prueba funcional. Su incorporación al proceso de integración continua ayuda a mantener criterios de calidad comunes entre los distintos componentes del proyecto [19].

3.4. Ingeniería de aplicaciones móviles

El desarrollo móvil enfrenta retos que no existen (o existen de forma distinta) en el desarrollo web: heterogeneidad de dispositivos y versiones de sistema operativo, restricciones de batería y de red intermitente, un ciclo de vida de la aplicación gestionado por el sistema operativo (y no por el desarrollador), y ventanas de publicación controladas por tiendas de aplicaciones. La decisión entre desarrollo nativo (Kotlin/Swift) y multiplataforma (Flutter, React Native) no es una preferencia estética: tiene consecuencias medibles en rendimiento, acceso a APIs del dispositivo, tamaño del binario y tiempo de llegada al mercado, razón por la cual la rúbrica exige justificar la elección con al menos dos criterios cuantitativos.

3.5. Integración y entrega continuas (CI/CD)

DevOps es una cultura de desarrollo de software que integra los equipos de desarrollo y operaciones mediante la automatización, la colaboración y la entrega continua de software. Uno de sus pilares es el uso de pipelines *CI/CD* (Continuous Integration y Continuous Delivery), los cuales permiten construir, probar y desplegar aplicaciones de manera repetible y confiable [13], [15].

3.5.1. Pipeline CI/CD

Un pipeline CI/CD está compuesto por una secuencia de etapas automatizadas que validan el software antes de su liberación. Las fases más comunes son:

- **Build:** compila el proyecto y resuelve dependencias.
- **Test:** ejecuta pruebas unitarias y de integración.
- **Lint:** analiza el código para detectar errores de estilo y calidad.
- **Publish:** genera y publica artefactos o imágenes Docker.
- **Deploy:** despliega automáticamente hacia un entorno de pruebas o producción.

La automatización de estas etapas reduce errores humanos y acelera la entrega de nuevas versiones [14].

3.5.2. GitHub Actions y sintaxis YAML

GitHub Actions implementa los pipelines mediante archivos YAML ubicados en el directorio `.github/workflows/`. Un workflow define los eventos que lo activan (**on**), los trabajos (**jobs**) y los pasos (**steps**) que ejecutarán acciones o comandos sobre un entorno virtual [13].

Los elementos principales de un workflow son:

- **Triggers:** eventos como `push`, `pull_request` o ejecución manual.
- **Jobs:** conjuntos de tareas que pueden ejecutarse en paralelo.
- **Steps:** instrucciones individuales, comandos o acciones reutilizables.
- **Marketplace Actions:** componentes reutilizables publicados por la comunidad para instalar dependencias, ejecutar pruebas o desplegar aplicaciones.

3.5.3. Estrategias de despliegue

Los pipelines modernos pueden implementar distintas estrategias de liberación para minimizar el riesgo durante una actualización:

- **Blue-Green:** mantiene dos entornos idénticos y redirige el tráfico al nuevo cuando es validado.
- **Canary:** libera la nueva versión únicamente a un pequeño porcentaje de usuarios antes del despliegue completo.
- **Rolling Update:** reemplaza progresivamente las instancias antiguas sin interrumpir el servicio.

Estas estrategias incrementan la disponibilidad y facilitan la recuperación ante fallos durante el despliegue [20].

3.5.4. Cobertura de código dentro del pipeline

La cobertura de código mide el porcentaje de instrucciones ejecutadas por las pruebas automatizadas. Integrar herramientas de cobertura en el pipeline permite verificar la calidad del software en cada confirmación de cambios y evitar que nuevas versiones reduzcan el nivel de pruebas existente [21].

3.6. Observabilidad distribuida

La observabilidad de un sistema distribuido se sustenta convencionalmente en tres señales complementarias: métricas (agregados numéricos con etiquetas, por ejemplo tasa de peticiones por ruta), registros o *logs* (eventos discretos, idealmente estructurados) y trazas (secuencias de *spans* correlacionados por un identificador común que permiten reconstruir el camino de una petición a través de varios servicios). El estándar OpenTelemetry busca unificar la instrumentación de estas tres señales de forma independiente del proveedor de observabilidad que las consuma después (Prometheus, Grafana, etc.).

3.7. Modelo de calidad ISO/IEC 25010:2023

El modelo ISO/IEC 25010:2023 organiza la calidad de un producto de software en ocho características — adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad — cada una descompuesta en subcaracterísticas medibles mediante métricas objetivas. Una evaluación seria bajo este modelo no describe la calidad de forma cualitativa, sino que fija un umbral objetivo por métrica y

reporta si el sistema lo alcanza, con el nivel de rigor de un experimento controlado (media, desviación típica e intervalo de confianza sobre repeticiones).

4. Arquitectura del sistema

4.1. Vistas C4, ADR de origen y evolución desde E1

Los diagramas C4 de nivel 1 y nivel 2 de la Entrega 1 (`docs/entrega1/entrega1.pdf`) representaban un sistema todavía no construido: actores genéricos (Cliente, Trabajador, Administrador), un backend único conectado a **una sola** instancia PostgreSQL, y PayPal en vez de la pasarela efectivamente integrada. Las tres decisiones arquitectónicas que E1 documentó como prosa — arquitectura de microservicios, uso de un API Gateway y motor de compatibilidad como servicio independiente — se recuperaron y formalizaron como ADR independientes en esta entrega (`docs/adr/ADR-009-arquitectura-microservicios.md`, `ADR-010-api-gateway.md` y `ADR-011-motor-compatibilidad-microservicio.md`), cada una con una sección explícita de qué se confirmó y qué se corrigió frente a lo que E1 planteó; en particular, ADR-011 documenta que el motor de compatibilidad previsto en E1 (programación por restricciones con co-clustering explicable) no es el que finalmente se construyó (**armado-ia**: reglas deterministas más un LLM para la explicación), aunque la justificación de aislarlo como microservicio propio se sostiene.

Las vistas C4 de nivel 1 (contexto) y nivel 2 (contenedores) construidas en E3 — frontend con Spring Cloud Gateway enrutando hacia los seis microservicios de Productos, Inventario, Pedidos, Órdenes y proveedores, Usuarios y Ventas — se mantienen vigentes en su forma general: esta entrega no agrega ni elimina microservicios, refactoriza su interior y asegura su conexión a la base de datos distribuida. El cambio relevante a nivel de contenedor es que **todos** los microservicios usan ahora CockroachDB como base real (Sección 4.4), mientras que en E3 esto era un perfil experimental paralelo a la base compartida usada para la validación funcional. La Figura 1 consolida esta vista actualizada: capa de clientes (SPA web y app Android), capa de acceso y seguridad (Caddy y el Gateway con filtro JWT), los seis microservicios y **armado-ia** sobre la red interna de Docker, el clúster CockroachDB de tres nodos con replicación Raft, y los dos servicios externos (SMTP para OTP, Amazon Bedrock para las explicaciones de **armado-ia**).

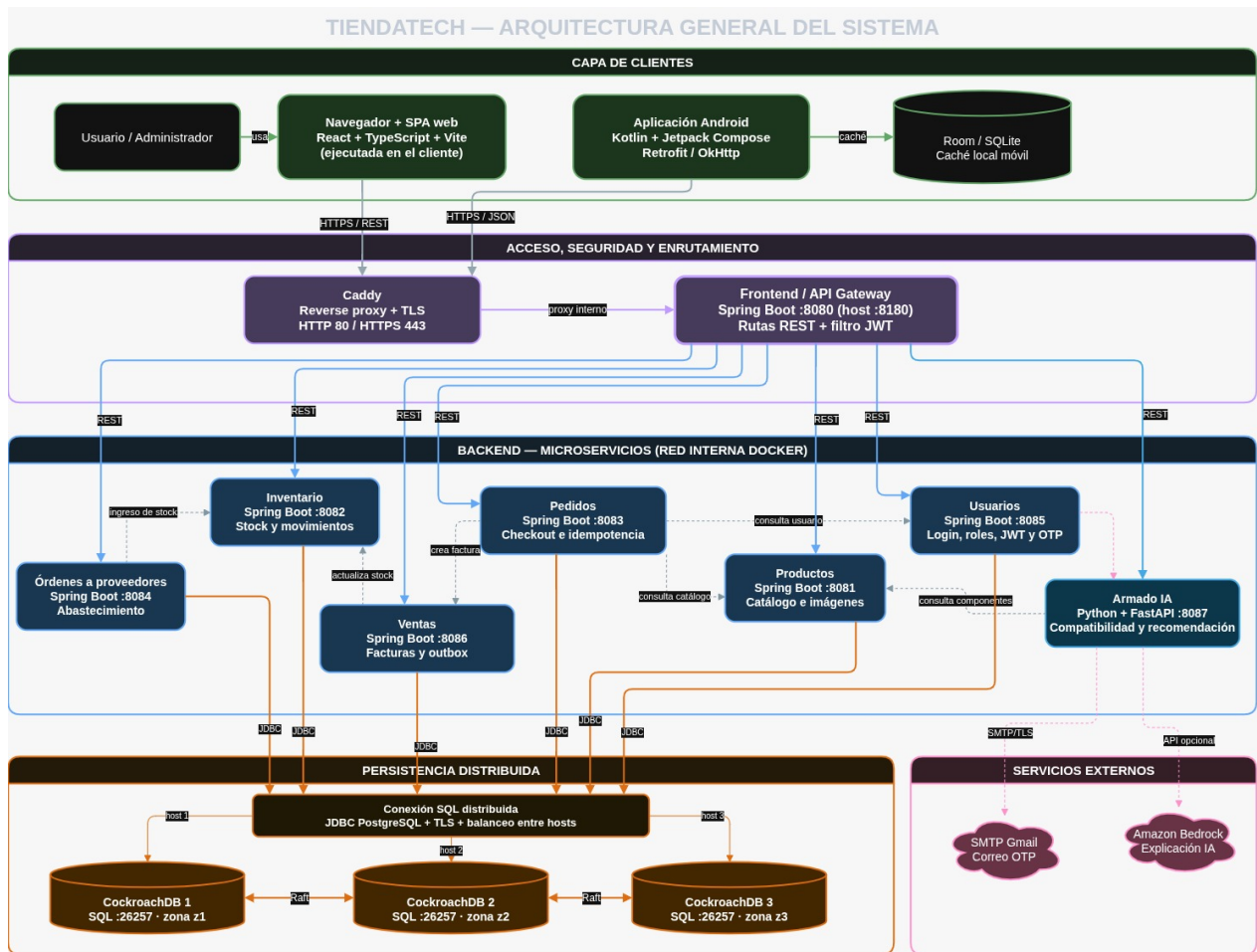


Figura 1: Arquitectura general de TiendaTech en la Entrega 4, con la persistencia distribuida ya unificada sobre CockroachDB.

Esta figura consolida en un único diagrama lo que formalmente corresponde a los niveles 1 y 2 de C4 más el despliegue físico sobre Docker. Queda declarada aquí, en vez de oculta, una limitación real: el repositorio conserva un diagrama de nivel 3 (`docs/diagrams/tiendatech-c4-l3-checkout.drawio.png`, flujo de *checkout*) y un diagrama de despliegue independiente (`docs/diagrams/tiendatech-despliegue.drawio.png`) de una etapa anterior del proyecto, y ambos muestran una única instancia PostgreSQL en vez del clúster CockroachDB de tres nodos con mTLS que el sistema usa realmente desde la Sección 4.4. Incluirlos sin corregirlos violaría el requisito explícito de esta entrega de documentar el sistema real y no el planificado; regenerarlos en `draw.io` contra la topología vigente queda registrado como pendiente técnico inmediato, no resuelto en silencio.

4.2. Refactor en capas y principios SOLID (D1.1)

Los seis microservicios Java del backend (`usuarios`, `productos-service`, `inventario-service`, `pedidos-service`, `ordenes-proveedores-service` y

`ventas-service`) quedaron organizados en los cuatro paquetes exigidos: `presentation` (controladores REST y DTO de entrada/salida), `application` (servicios de caso de uso, sin acceso directo a JDBC ni a HTTP saliente), `domain` (entidades y *puertos*, es decir, interfaces que declaran lo que el dominio necesita del exterior) e `infrastructure` (adaptadores concretos: repositorios JDBC, clientes HTTP, configuración de seguridad). El microservicio `armado-ia`, escrito en Python, mantiene una separación equivalente por responsabilidad (`clients/`, `explicacion/`, capa de servicio) sin adoptar literalmente la misma nomenclatura de paquetes Java.

El patrón de puerto y adaptador se aplica de forma consistente: `ordenes-proveedores-service` y `ventas-service` declaran `InventarioPort` en `domain`, implementado por `InventarioClient` en `infrastructure`; `ventas-service` añade además `FacturaStore` y `FacturaOutboxStore` como puertos de persistencia; `pedidos-service` es el caso más extenso, con puertos dedicados para cada colaborador externo (`ProductoPort`, `UsuarioPort`, `FacturaPort`, `CrdbProbePort`, `CrdbRetryPort`, `CrdbMetricsPort`, `BusinessMetricsPort`); y `usuarios` generaliza la convención a `domain/port/out` (`TokenPort`, `UsuarioRepositoryPort`, `EmailPort`, `PasswordHasher`, `OtpStorePort`, entre otros), cada uno con su adaptador en `infrastructure/persistence/adapter` o `infrastructure/security`.

Esta organización materializa dos principios SOLID de forma verificable en el código. El **principio de responsabilidad única** (SRP) separa qué hace el sistema (`application`) de cómo lo hace (`infrastructure`): por ejemplo, `OrdenCompraService` orquesta la recepción de una orden de compra sin saber si el inventario se actualiza vía HTTP, cola de mensajes o llamada local. El **principio de inversión de dependencias** (DIP) se sostiene en que las clases de `application` dependen únicamente de las interfaces declaradas en `domain`, nunca de una clase concreta de `infrastructure`; Spring resuelve la implementación real (`InventarioClient`, `JdbcOrdenCompraRepository`, etc.) por inyección en tiempo de arranque. Esto permite, por ejemplo, sustituir `InventarioClient` por un doble de prueba sin tocar `OrdenCompraService`.

4.3. Patrones GoF aplicados (D1.2)

La documentación completa de esta decisión vive en `docs/adr/ADR-005-patrones-gof.md`. La auditoría registrada en ese ADR es deliberadamente conservadora: de los ocho servicios del backend, la mayoría implementa una arquitectura por capas limpia sin patrones GoF genuinos, y lo que a primera vista parece un patrón resulta ser inyección de dependencias de Spring, un *Repository* (Fowler, no GoF) con una sola implementación, o un *builder* de una librería de terceros. En vez de etiquetar falsamente esos casos, el equipo documenta cinco patrones GoF con evidencia real en el código:

1. **Strategy** (comportamiento): `ExplicacionClient` (`services/armado-ia/app/explicacion/client.py`) como interfaz, con dos estrategias intercambiables — `BedrockExplicacionClient` (usa AWS Bedrock) y `DeterministicExplicacionClient` (redacción sin LLM) — seleccionadas en runtime por `ExplicacionService`.

2. **Adapter** (estructural): `BedrockExplicacionClient` envuelve el SDK `boto3` y `ProductoClient` (`services/armado-ia/app/clients/producto_client.py`) envuelve `httpx`, ambos traduciendo su fuente externa a tipos de dominio propios.
3. **Decorator** (estructural): `TrustedUserHeaderRequest`, clase interna de `JwtGatewayFilter` (`Apps/web/frontend`), extiende `HttpServletRequestWrapper` para inyectar cabeceras de identidad verificadas sin mutar la petición original.
4. **Chain of Responsibility** (comportamiento): `ExplanationValidationHandler` y sus implementaciones concretas (`NonEmptyExplanationHandler`, `VerifiedHardwareTokenHandler`) forman una cadena de validadores anti-alucinación sobre la respuesta del LLM.
5. **Template Method** (comportamiento): el método `validate()` de `ExplanationValidationHandler` fija el algoritmo común (ejecutar regla, devolver problema si existe, delegar al siguiente) y delega el paso variable a `_validate_current()` en cada subclase.

El ADR también deja constancia explícita de los patrones evaluados y descartados por no tener estructura GoF real en este backend (Strategy sobre puertos con una sola implementación, Factory Method/Abstract Factory sobre `@Bean` sin jerarquía Creator, Builder de terceros, Observer, Facade, Command, State y Singleton), y documenta por separado — sin mezclarlos con el catálogo GoF — otros patrones legítimos presentes en el proyecto: *Repository* (Fowler) en los seis servicios Java, *Outbox* en `ventas-service` y *Circuit Breaker/Retry* (Nygard) en `pedidos-service`, `ordenes-proveedores-service` y `armado-ia`.

4.4. Persistencia distribuida operando (D4.2)

El clúster CockroachDB de tres nodos heredado de la Entrega 3 (puertos 26257/26258/26259) se mantiene operando y es el que sostiene la persistencia de los seis microservicios Java. Sobre esa base, esta entrega incorpora dos mejoras verificables en `docker-compose.yml`.

La primera es de **disponibilidad**: los siete servicios que dependen de una base de datos o exponen un endpoint HTTP (`pedidos`, `productos`, `inventario`, `ventas`, `ordenes-proveedores`, `usuarios` y `armado-ia`) ahora declaran un bloque `healthcheck`: contra `/actuator/health` (o el equivalente del servicio Python), con reintentos y periodo de arranque configurados; antes de esta entrega, `tiendatech-ventas` y `tiendatech-ordenes-proveedores` carecían de este chequeo, lo que impedía que `docker compose` reportara con precisión cuándo el sistema completo estaba realmente listo para recibir tráfico.

La segunda es de **seguridad de la conexión**: la cadena de conexión de los seis servicios Java se migró de `jdbc:postgresql://<IP fija de AWS>:26257,...&sslmode=disable` — una conexión sin cifrar contra una dirección IP fija — a `jdbc:postgresql://host.docker.internal:26257,...&sslmode=verify-ca&sslrootcert=/app/crdb-certs/ca.crt&sslcert=/app/crdb-certs/client.jhinson.crt&sslkey=/app/crdb-certs/client.jhinson.key.pk8`, es decir, autenticación mutua por certificado

(mTLS) en vez de una conexión abierta. Los tres archivos de certificado se montan como volúmenes de solo lectura (`./Apps/web/infra/crdb-certs/*.crt|.key.pk8:...:ro`) en cada contenedor. Este cambio requirió reconciliar manualmente un conflicto de fusión en `docker-compose.yml` porque ambas mejoras (healthchecks y mTLS) se desarrollaron en paralelo sobre los mismos bloques de servicio; el resultado final conserva ambas.

4.5. Stack tecnológico

La Tabla 2 consolida las versiones vigentes verificadas directamente en los archivos de build del repositorio (`pom.xml`, `package.json`, `libs.versions.toml`), actualizando la Tabla de herramientas de `docs/entrega3/PFC3.tex` con lo efectivamente usado en esta entrega.

Cuadro 2: Stack tecnológico consolidado por capa (Entrega 4).

Capa	Tecnología	Versión verificada
Backend (6 microservicios Java)	Java / Spring Boot	Java 21, Spring Boot 3.3.5 (<code>pom.xml</code> de <code>pedidos-service</code>)
Backend (armado-ia)	Python	FastAPI, pytest + ruff en CI
Persistencia distribuida	CockroachDB	v23.2.4 verificado en la prueba de integración con Testcontainers de <code>pedidos-service</code> ; v23.1.13 en el nodo local de <code>docker- compose.override.yml</code>
Web (SPA)	React / TypeScript	React 18.3.1, <code>react-router-dom</code> 7.18.2, TypeScript 5.9.2, Vite 7.1.2 (<code>package.json</code>)
Web (Gateway)	Spring Cloud Gateway	Empaquetado junto al frontend, ver <code>Dockerfile</code> multi-stage (Sección 5.2)
Móvil	Kotlin / Jetpack Compose	Kotlin 2.2.10, Compose BOM 2026.02.01, Hilt 2.60.1, <code>compileSdk/targetSdk</code> 37, <code>minSdk</code> 26 (<code>libs.versions.toml</code> , <code>build.gradle.kts</code>)
Contenerización	Docker / Docker Compose	Build multi-stage por servicio; publicación multi-arquitectura <code>linux/a md64,linux/arm64</code> en <code>publish-images.yml</code>
CI/CD	GitHub Actions	<code>ci.yml</code> (3 jobs) + <code>publish-images.yml</code> , ver Sección 7.3

5. Aplicación web

5.1. Funcionalidad (D2.1)

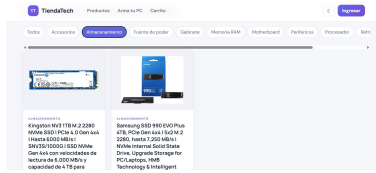
La SPA (`Apps/web/frontend/webapp`), construida con React 18 y `react-router-dom`, expone doce rutas declaradas en `App.tsx`: catálogo (`/`), detalle de producto (`/producto/:id`), armado de PC (`/armado`), carrito (`/carrito`), pago (`/pago`), factura (`/factura/:id?`), autenticación (`/login`, `/registro`, `/recuperacion`), cuenta (`/cuenta`), panel de trabajador (`/trabajador`) y administración (`/admin`), superando ampliamente el mínimo de cinco exigido por la rúbrica.

La sesión se identifica mediante un token JWT emitido por `tiendatech-usuarios`. El módulo `src/services/session.ts` guarda el token exclusivamente en `sessionStorage` (nunca en `localStorage`: al cargar, el propio módulo purga activamente cualquier clave de sesión que hubiera quedado en `localStorage` de una versión anterior), cumpliendo la restricción explícita de la rúbrica sobre el manejo del token. La ruta `/admin` está protegida por un componente `AdminRoute` que valida el rol del usuario (`id_rol === 1` o `rol === "admin"`) y redirige a `/login` si no se cumple. Adicionalmente, `App.tsx` implementa expiración de sesión por inactividad (30 minutos sin interacción del usuario cierran la sesión automáticamente).

La internacionalización declarada como deseable en la rúbrica (`react-i18next` con al menos español e inglés) no está implementada: toda la interfaz está codificada directamente en español. Se documenta aquí como brecha conocida en vez de omitirla.



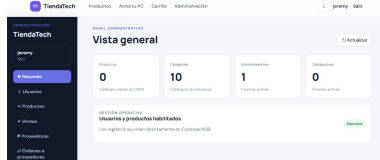
(a) Página principal (/)



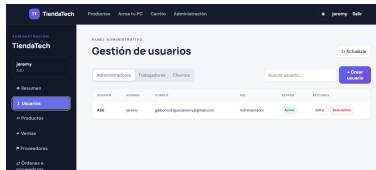
(b) Catálogo filtrado por categoría



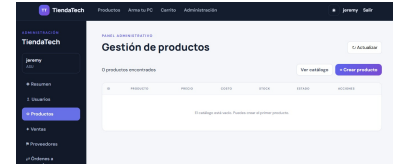
(c) Armado de PC (/armado)



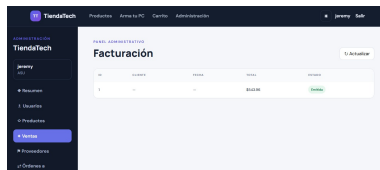
(d) Panel administrativo, resumen (/admin)



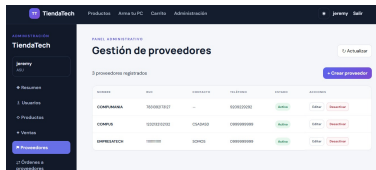
(e) Administración de usuarios



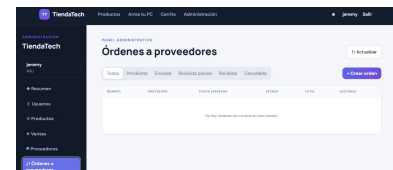
(f) Administración de productos



(g) Facturación



(h) Administración de proveedores



(i) Órdenes a proveedores

Figura 2: Capturas de las rutas principales de la SPA (página principal, catálogo, armado de PC) y del panel de administración (/admin), que cubre las seis entidades de negocio del sistema. La interfaz está codificada íntegramente en español, consistente con la ausencia de `react-i18next` señalada arriba.

5.2. Calidad (D2.2)

El `package.json` de la webapp no declara `eslint` ni `prettier` como dependencias de desarrollo, por lo que no hay una verificación de estilo o de reglas de código automatizada más allá de la que impone el propio compilador de TypeScript (`tsc -b`) durante el paso de `build`. Tampoco hay un framework de pruebas configurado (ni Vitest ni Jest aparecen en `devDependencies`), por lo que no existe cobertura de pruebas automatizada para la SPA en este momento; esta ausencia se retoma como brecha en la Sección 10.

El empaquetado sí está resuelto correctamente: `Apps/web/frontend/Dockerfile` usa una construcción *multi-stage* de tres etapas — `node:22-alpine` para compilar la SPA con `npm run build`, `maven:3.9-eclipse-temurin-17` para empaquetar el frontend Spring Cloud Gateway junto con los estáticos ya compilados de React, y `eclipse-temurin:17-jre` como imagen final de ejecución — lo que mantiene la imagen de producción libre de herramientas de compilación.

6. Aplicación móvil

6.1. Funcionalidad (D3.1)

`Apps/mobile` es una aplicación Android nativa en Kotlin con Jetpack Compose, organizada por *feature* (`auth`, `catalog`, `cart`, `orders`, `account`, `scanner`, `notifications`), cada una dividida internamente en `data/domain/ui`, con inyección de dependencias vía Hilt y comunicación HTTP con el backend mediante Retrofit y OkHttp. La capa `core/database` usa Room para cachear productos y categorías localmente (`ProductCacheDao`, `CategoryCacheDao`, `CacheMetadataDao`), permitiendo operar el catálogo con datos ya vistos aunque la conexión falle.

La aplicación utiliza dos capacidades del dispositivo, superando el mínimo exigido por la rúbrica. La primera es la **cámara**: la *feature scanner* usa `CameraX` (`androidx.camera.core.camera2/lifecycle/view`) junto con ML Kit Barcode Scanning para leer códigos de barras de productos y resolverlos contra el catálogo (`BarcodeAnalyzer`, `BarcodeLookup`). La segunda son las **notificaciones nativas**: `AndroidOrderNotificationPublisher` publica notificaciones reales del sistema operativo (`API NotificationManager/NotificationCompat`, con el permiso `POST_NOTIFICATIONS` declarado en el manifiesto) para informar cambios de estado en las órdenes del usuario.

Los flujos de negocio cubiertos son autenticación (`login/registro`, *feature/auth*), catálogo y detalle de producto (*feature/catalog*), carrito (*feature/cart*), órdenes y su historial (*feature/orders*), y cuenta con checkout (*feature/account*, incluye `CheckoutScreen`). La sesión se protege de forma más estricta que en la web: el token no se guarda en texto plano sino cifrado mediante Android Keystore (`AndroidKeystoreTokenCipher`, `SecureSessionStore`), y `AuthInterceptor` lo adjunta automáticamente a las peticiones salientes de Retrofit.

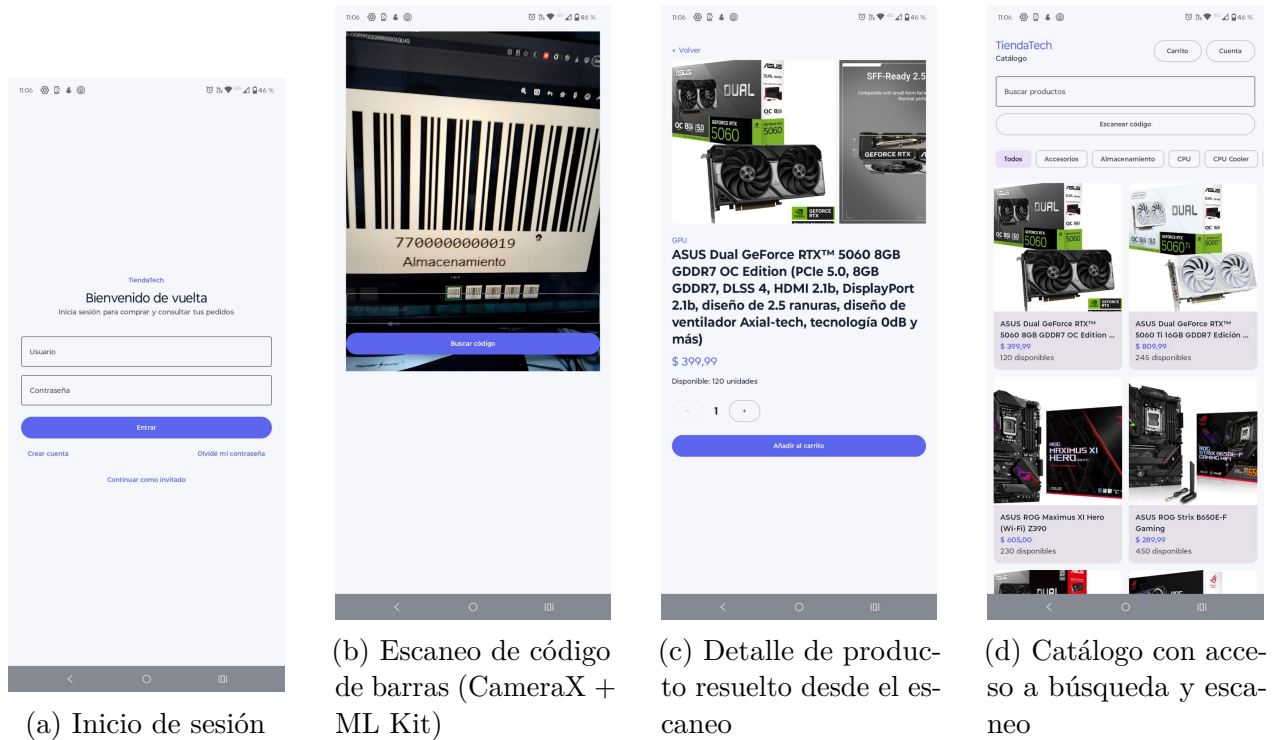


Figura 3: Flujo de autenticación y evidencia visual de la capacidad de cámara (escaneo de código de barras vía CameraX y ML Kit) en la aplicación móvil nativa. La segunda capacidad usada, notificaciones nativas (`AndroidOrderNotificationPublisher`), no se ilustra aquí para evitar exponer datos personales reales de la cuenta de prueba visibles en esa pantalla.

6.2. Calidad (D3.2)

La aplicación cuenta con dos niveles de prueba verificados en el repositorio. Existen pruebas instrumentadas (`app/src/androidTest`) que corren sobre un dispositivo o emulador real: `TiendaTechDatabaseTest` valida el esquema Room y `NotificationIntegrationTest` valida la publicación de notificaciones nativas, además de la plantilla `ExampleInstrumentedTest` generada por Android Studio. Adicionalmente — y a diferencia de lo que se había señalado como pendiente de confirmar en la auditoría interna previa a esta entrega — sí existen pruebas unitarias reales sobre `app/src/test`: doce archivos que cubren `ViewModels` (`ScannerViewModelTest`, usando `kotlinx.coroutines.test` con `StandardTestDispatcher` y `runTest` para probar corrutinas de forma determinista), repositorios (`AuthRepositoryTest`, `CartContractTest`, `OrdersContractTest`), validadores (`AuthValidatorTest`), mapeadores (`CatalogMapperTest`) y almacenamiento seguro de sesión (`SecureSessionStoreTest`), todas con JUnit 4 y `okhttp-mockwebserver` para simular el backend.

El job `android-mobile` de `.github/workflows/ci.yml` ejecuta `lintDebugtestDebugUnitTestassembleDebugassembleDebugAndroidTest` en cada push y publica el APK de depuración y el APK de pruebas instrumentadas como artefactos descargables del pipeline, cumpliendo la exigencia de que el CI produzca un artefacto instalable de la app móvil.

7. Pruebas y CI/CD

7.1. Pirámide de pruebas (D5.1)

A la fecha de cierre de esta sección, los seis microservicios Java cuentan con pruebas unitarias reales con JUnit 5 y superan el umbral mínimo de cobertura del 70 % exigido por la rúbrica, verificado automáticamente por `jacoco-maven-plugin` (objetivo `check`, ligado a la fase `test`, con regla `BUNDLE/LINE/COVEREDRATIO ≥ 0,70`) sobre el paquete `application` de cada servicio. `ordenes-proveedores-service` y `ventas-service` — que en un estado previo de este documento no tenían ninguna prueba — incorporaron `OrdenCompraServiceTest`, `ProveedorServiceTest`, `FacturaServiceTest` e `InventarioOutboxProcessorTest` respectivamente, cerrando la brecha señalada anteriormente sobre la lógica de negocio más nueva de esta entrega (recepción parcial de órdenes de compra, recálculo de costo por promedio ponderado, auto-desactivación de productos). El microservicio Python `armado-ia` aplica el mismo criterio con `pytest-cov` (`-cov-fail-under=70`). La Tabla 3 resume la cobertura medida por servicio.

Cuadro 3: Cobertura de pruebas por microservicio (Entrega 4), verificada por el gate automático del pipeline (mínimo 70 %).

Servicio	Cobertura (líneas)
<code>inventario-service</code>	100 %
<code>productos-service</code>	100 %
<code>ordenes-proveedores-service</code>	92.16 %
<code>ventas-service</code>	100 %
<code>pedidos-service</code>	80.62 %
<code>usuarios</code>	75.80 %
<code>armado-ia</code> (Python)	75 %

Además de las pruebas unitarias, `pedidos-service` incluye la única prueba de **integración** real con base de datos del backend: `CockroachDbIntegrationTest`, que levanta un contenedor `cockroachdb/cockroach:v23.2.4` real vía `Testcontainers` (`@Testcontainers`, `@Container`) en lugar de simular la base de datos. A esto se suma `GatewayIntegrationTest` (11 pruebas, 0 fallos), que levanta el API Gateway completo (`@SpringBootTest` con puerto aleatorio) contra tres backends simulados y verifica tres flujos reales atravesando el Gateway (catálogo de productos, ubicaciones/usuarios, creación de pedidos), más el rechazo con `401` sin token JWT en las ocho familias de rutas protegidas — cumpliendo así el mínimo de tres flujos de integración por Gateway exigido por la rúbrica.

Los Listados 1–3 ilustran, con código real del repositorio, los tres niveles de la pirámide efectivamente presentes en el backend: prueba unitaria pura, prueba de integración con base de datos real y prueba de integración a través del API Gateway.

Listing 1: Prueba unitaria pura, sin colaboradores externos (inventario-service).

```
package com.tiendatech.inventario.application.reservation;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class LamportClockTest {
    @Test
    void avanzaElRelojAlMaximoEntreLocalYRemotoMasUno() {
        var clock = new LamportClock();
        assertEquals(0, clock.current());
        assertEquals(6, clock.receive(5));
        assertEquals(6, clock.current());
        assertEquals(7, clock.receive(3));
        assertEquals(7, clock.current());
        assertEquals(20, clock.receive(19));
        assertEquals(20, clock.current());
    }
}
```

Listing 2: Prueba de integración con base de datos real vía Testcontainers (pedidos-service).

```
package com.tiendatech.pedidos.integration;

import org.junit.jupiter.api.Test;
import org.testcontainers.cockroachdb.CockroachContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

import static org.junit.jupiter.api.Assertions.assertEquals;

@Testcontainers(disabledWithoutDocker = true)
class CockroachDbIntegrationTest {

    @Container
    static final CockroachContainer cockroach =
        new CockroachContainer("cockroachdb/cockroach:v23.2.4")
            .withDatabaseName("tiendatech_test");

    @Test
    void ejecutaTransaccionSerializableConDriverPostgresql() throws Exception
    {
```

```

        try (Connection connection = DriverManager.getConnection(
            cockroach.getJdbcUrl(),
            cockroach.getUsername(),
            cockroach.getPassword())) {
            connection.setTransactionIsolation(Connection.
TRANSACTION_SERIALIZABLE);
            connection.setAutoCommit(false);

            try (Statement statement = connection.createStatement()) {
                statement.execute("""
                    CREATE TABLE pedido_prueba (
                        fecha DATE NOT NULL,
                        orden_id INT8 NOT NULL,
                        total DECIMAL(18,2) NOT NULL,
                        PRIMARY KEY (fecha, orden_id)
                    )
                """);
                statement.executeUpdate("""
                    INSERT INTO pedido_prueba (fecha, orden_id, total)
                    VALUES (DATE '2026-07-28', 1, 125.50)
                """);
            }
            connection.commit();

            try (Statement statement = connection.createStatement();
                ResultSet result = statement.executeQuery(
                    "SELECT total FROM pedido_prueba WHERE orden_id = 1")
            ) {
                result.next();
                assertEquals("125.50", result.getBigDecimal("total").
toPlainString());
            }
        }
    }
}

```

Listing 3: Extracto de la prueba de integración a través del API Gateway (Apps/web/frontend); se omiten por brevedad los métodos auxiliares que levantan los backends simulados.

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class GatewayIntegrationTest {
    private static final String SECRET = "
TiendaTechDistribuidaJwtSecretKey2026Seguro";
    private static final HttpServer PRODUCTOS = server("/api/productos/10", "
{"servicio\":\"productos\",\"id\":10}");
    private static final HttpServer USUARIOS = server("/api/provincias", "[{"

```

```

id\:1,\"nombre\":"Guayas\"}]]");
private static final HttpServer PEDIDOS = server("/api/ordenes", "{\"
servicio\":"pedidos\", \"creada\":true}");

@LocalServerPort int gatewayPort;

@dynamicPropertySource
static void backends(DynamicPropertyRegistry registry) {
    registry.add("tiendatech.security.jwt.secret", () -> SECRET);
    registry.add("PRODUCTOS_SERVICE_URL", () -> url(PRODUCTOS));
    registry.add("USUARIOS_SERVICE_URL", () -> url(USUARIOS));
    registry.add("PEDIDOS_SERVICE_URL", () -> url(PEDIDOS));
    registry.add("tiendatech.security.jwt.public-read-paths", () -> "/api/
productos/**,/api/provincias/**");
}

@Test void flujoCatalogoAtraviesaGatewayHastaProductos() throws Exception
{
    var response = get("/api/productos/10");
    assertEquals(200, response.statusCode()); assertTrue(response.body().
contains("productos"));
}

@Test void flujoUbicacionesAtraviesaGatewayHastaUsuarios() throws
Exception {
    var response = get("/api/provincias");
    assertEquals(200, response.statusCode()); assertTrue(response.body().
contains("Guayas"));
}

@Test void flujoPedidosAtraviesaGatewayHastaElMicroservicio() throws
Exception {
    var response = get("/api/ordenes", token());
    assertEquals(200, response.statusCode()); assertTrue(response.body().
contains("creada"));
}

@ParameterizedTest
@ValueSource(strings = {
    "/api/usuarios/buscar", "/api/movimientos", "/api/carrito/1",
    "/api/ordenes", "/api/proveedores", "/api/ordenes-compra",
    "/api/facturas/1", "/api/armado/recomendar"
})
void todaFamiliaProtegidaRechazaSolicitudSinToken(String path) throws
Exception {
    assertEquals(401, get(path).statusCode(), path);
}
// ... helpers de infraestructura de prueba (get, token, gateway, server,

```

```
    respond) omitidos
}
```

No existe una carpeta `tests/` en la raíz del repositorio, por lo que tampoco hay pruebas end-to-end de la web con Playwright ni pruebas de carga con Locust; esta última queda como paso inmediato siguiente, al ser un insumo compartido con la evaluación ISO/IEC 25010 (Sección 9). La aplicación móvil sí cierra su propia pirámide de forma más completa, con pruebas unitarias y pruebas instrumentadas (detalladas en la Sección 6.2).

7.2. Contratos de consumidor Pact (D4.1)

Como capa de *contrato* de la pirámide de pruebas, la rúbrica exige *consumer-driven contract testing* con Pact entre los clientes (web y móvil, como consumidores) y el backend (como proveedor). A la fecha de cierre de esta sección no existe en el repositorio ninguna carpeta `tests/contract` ni configuración de Pact (no hay dependencias de `pact-jvm` en ningún `pom.xml` del backend, ni de un *Pact broker* referenciado en el pipeline de CI/CD). Este criterio se declara explícitamente como **no cumplido** en el estado actual del proyecto. La razón no es desconocimiento de la herramienta, sino una decisión de priorización bajo el tiempo disponible del equipo: se optó por asegurar primero que la persistencia distribuida operara de forma segura (Sección 4.4) y que la calidad de las aplicaciones cliente avanzara (Secciones 5 y 6), antes de introducir una herramienta de verificación de contratos que ninguno de los servicios consumidores tenía previamente. Queda como trabajo inmediato siguiente, con una ruta ya decidida en `docs/plan-cierre-e4.md`: priorizar el contrato *web*↔*backend* (el cliente con más tráfico y más rutas) antes que el contrato del consumidor móvil, de modo que exista al menos un contrato verificado en verde en lugar de dos a medio terminar.

7.3. Pipeline CI/CD end-to-end (D5.2)

El repositorio incorporó en este ciclo un segundo workflow de GitHub Actions, `.github/workflows/ci-cd.yml` (*CI-CD quality gate*), que corre en cada *push* y *pull request* contra `main` (además de `workflow_dispatch` manual), con `concurrency` configurado para cancelar ejecuciones obsoletas del mismo *ref*. Define seis jobs: `test-java` (matriz sobre los seis microservicios, ejecuta `mvn clean test` y aplica el gate de JaCoCo de la Tabla 3), `test-python` (pytest con cobertura sobre `armado-ia`), `lint` (compilación de los seis servicios Java, `ruff` sobre `armado-ia` y build de TypeScript sobre la SPA), `integration-test` (`GatewayIntegrationTest` y `CockroachDbIntegrationTest`), `build` (matriz de `docker/build-push-action` que construye, sin publicar, las ocho imágenes del sistema con cacheo `type=gha`) y `coverage` (publica los reportes XML de JaCoCo y `pytest-cov` a Codecov). El job `build` depende explícitamente de que `test-java`, `test-python`, `lint` e `integration-test` hayan pasado (`needs`), de modo que no se construye ninguna imagen sobre código que no haya superado su quality gate. El workflow `.github/workflows/ci.yml` original se mantiene activo en paralelo

(jobs `android-mobile`, `crdb-tests`, `armado-ia-tests`); su consolidación con `ci-cd.yml` en un único pipeline queda pendiente como limpieza técnica, declarada aquí en vez de resuelta en silencio.

Comparado contra los siete jobs que exige la rúbrica (`lint`, `test-backend`, `test-web`, `test-mobile`, `build-images`, `build-mobile-apk`, `integration`), el pipeline vigente cubre ahora `test-backend` (`test-java`, sobre los seis servicios y no solo `pedidos-service`), `integration` (`integration-test`, con dos flujos — Gateway y base de datos — en vez de uno), `build-images` (job build de `ci-cd.yml`, además de `publish-images.yml`), `test-mobile` y `build-mobile-apk` (ambos dentro de `android-mobile`, sin cambios respecto de la entrega anterior), y `lint` de forma parcial (compilación Java y `ruff` sí corren; no hay un linter de estilo dedicado tipo Checkstyle/ESLint, solo un build de TypeScript). Sigue sin job dedicado `test-web`, porque, como se documentó en la Sección 5.2, la SPA no tiene ningún framework de pruebas configurado. Como en la entrega anterior, `publish-images.yml` publica en Docker Hub y no en GHCR; el resultado funcional es equivalente.

La guía específica del Paso 2 (*Pipeline CI/CD con GitHub Actions*) describe la misma exigencia con una nomenclatura más compacta, de cuatro jobs: `test` (ejecuta pruebas y cobertura), `lint` (análisis estático), `build` (construye la imagen Docker) e `integration-test` (levanta el entorno y ejecuta las pruebas de integración). La correspondencia con los seis jobs reales de `ci-cd.yml` es directa: `test` → `test-java` + `test-python` + `coverage` (pruebas y cobertura de los seis servicios Java y de `armado-ia`, con la publicación del reporte incluida); `lint`, `build` e `integration-test` conservan el mismo nombre. Se optó por mantener la separación en seis jobs — en vez de fusionar `test-java`, `test-python` y `coverage` hasta calcar un único job `test` — porque el pipeline ya estaba probado y en verde con esta estructura; fusionarlos habría exigido volver a validar el pipeline completo bajo la misma restricción de tiempo documentada en la Discusión (Sección 10), a cambio de un beneficio puramente nominal. El Listado 4 reproduce el archivo completo tal como quedó configurado y verificado en verde.

```
1 name: CI-CD quality gate
2
3 on:
4   push:
5     branches: [main]
6   pull_request:
7     branches: [main]
8   workflow_dispatch:
9
10 permissions:
11   contents: read
12
13 concurrency:
14   group: ci-cd-${{ github.ref }}
15   cancel-in-progress: true
16
17 jobs:
18   test-java:
```

```

19  name: Test ${matrix.service}
20  runs-on: ubuntu-latest
21  timeout-minutes: 25
22  strategy:
23    fail-fast: false
24    matrix:
25      service:
26        - inventario-service
27        - productos-service
28        - ordenes-proveedores-service
29        - ventas-service
30        - pedidos-service
31        - usuarios
32  steps:
33    - uses: actions/checkout@v4
34    - uses: actions/setup-java@v4
35      with:
36        distribution: temurin
37        java-version: "21"
38        cache: maven
39        cache-dependency-path: services/${matrix.service}/pom.xml
40    - name: Unit tests and JaCoCo quality gate
41      working-directory: services/${matrix.service}
42      run: mvn --batch-mode --no-transfer-progress clean test
43    - name: Preserve test and coverage reports
44      if: always()
45      uses: actions/upload-artifact@v4
46      with:
47        name: reports-${matrix.service}
48        path: |
49          services/${matrix.service}/target/surefire-reports/
50          services/${matrix.service}/target/site/jacoco/jacoco.xml
51        if-no-files-found: warn
52
53  test-python:
54    name: Test armado-ia
55    runs-on: ubuntu-latest
56    timeout-minutes: 15
57    steps:
58      - uses: actions/checkout@v4
59      - uses: actions/setup-python@v5
60        with:
61          python-version: "3.12"
62          cache: pip
63          cache-dependency-path: |
64            services/armado-ia/requirements.txt

```

```

65         services/armado-ia/requirements-dev.txt
66     - name: Install dependencies
67       working-directory: services/armado-ia
68       run: pip install -r requirements.txt -r requirements-dev.txt
69     - name: Unit tests and coverage gate
70       working-directory: services/armado-ia
71       run: python -m pytest --junitxml=pytest-junit.xml --cov=app.
           domain --cov=app.explicacion --cov-report=xml:coverage.
           xml --cov-fail-under=70
72     - uses: actions/upload-artifact@v4
73       if: always()
74       with:
75         name: reports-armado-ia
76         path: |
77           services/armado-ia/pytest-junit.xml
78           services/armado-ia/coverage.xml
79         if-no-files-found: warn
80
81 lint:
82   name: Static analysis
83   runs-on: ubuntu-latest
84   timeout-minutes: 20
85   steps:
86     - uses: actions/checkout@v4
87     - uses: actions/setup-java@v4
88       with:
89         distribution: temurin
90         java-version: "21"
91         cache: maven
92     - uses: actions/setup-python@v5
93       with:
94         python-version: "3.12"
95         cache: pip
96         cache-dependency-path: services/armado-ia/requirements-dev
           .txt
97     - uses: actions/setup-node@v4
98       with:
99         node-version: "22"
100         cache: npm
101         cache-dependency-path: Apps/web/frontend/webapp/package-
           lock.json
102     - name: Check Java compilation
103       shell: bash
104       run: |
105         set -euo pipefail
106         for service in inventario-service productos-service
           ordenes-proveedores-service ventas-service pedidos-
           service usuarios; do

```

```

107         mvn -q -f "services/$service/pom.xml" -DskipTests
108         compile
109     done
110 - name: Ruff static analysis
111     working-directory: services/armado-ia
112     run: |
113         pip install -r requirements.txt -r requirements-dev.txt
114         python -m ruff check app tests
115 - name: TypeScript static analysis
116     working-directory: Apps/web/frontend/webapp
117     run: |
118         npm ci
119         npm run build
120 integration-test:
121     name: Gateway and CockroachDB integration
122     runs-on: ubuntu-latest
123     timeout-minutes: 25
124     steps:
125     - uses: actions/checkout@v4
126     - uses: actions/setup-java@v4
127       with:
128         distribution: temurin
129         java-version: "21"
130         cache: maven
131     - name: Three flows through API Gateway
132       working-directory: Apps/web/frontend
133       run: mvn --batch-mode --no-transfer-progress -Dtest=
134         GatewayIntegrationTest test
135     - name: CockroachDB integration through Testcontainers
136       working-directory: services/pedidos-service
137       run: mvn --batch-mode --no-transfer-progress -Dtest=
138         CockroachDbIntegrationTest -Djacoco.skip=true test
139 build:
140     name: Build image ${ matrix.image }
141     needs: [test-java, test-python, lint, integration-test]
142     runs-on: ubuntu-latest
143     timeout-minutes: 35
144     strategy:
145       fail-fast: false
146     matrix:
147       include:
148       - { image: gateway, context: Apps/web/frontend, dockerfile
149         : Apps/web/frontend/Dockerfile }
150       - { image: usuarios, context: services/usuarios,
151         dockerfile: services/usuarios/Dockerfile }
152       - { image: productos, context: services/productos-service,

```

```

150         dockerfile: services/productos-service/Dockerfile }
151     - { image: inventario, context: ., dockerfile: services/
      inventario-service/Dockerfile }
152     - { image: pedidos, context: ., dockerfile: services/
      pedidos-service/Dockerfile }
153     - { image: ordenes-proveedores, context: services/ordenes-
      proveedores-service, dockerfile: services/ordenes-
      proveedores-service/Dockerfile }
154     - { image: ventas, context: services/ventas-service,
      dockerfile: services/ventas-service/Dockerfile }
155     - { image: armado-ia, context: services/armado-ia,
      dockerfile: services/armado-ia/Dockerfile }
156
157 steps:
158   - uses: actions/checkout@v4
159   - uses: docker/setup-buildx-action@v3
160   - name: Build without publishing
161     uses: docker/build-push-action@v6
162     with:
163       context: ${ matrix.context }
164       file: ${ matrix.dockerfile }
165       push: false
166       tags: tiendatech/${ matrix.image }:${ matrix.github.sha }
167       cache-from: type=gha,scope=${ matrix.image }
168       cache-to: type=gha,mode=max,scope=${ matrix.image }
169
170 coverage:
171   name: Publish coverage
172   needs: [test-java, test-python]
173   runs-on: ubuntu-latest
174   steps:
175     - uses: actions/checkout@v4
176     - uses: actions/download-artifact@v4
177     with:
178       pattern: reports-*
179       path: reports
180       merge-multiple: false
181     - name: Upload XML reports to Codecov
182       uses: codecov/codecov-action@v5
183       with:
184         token: ${ secrets.CODECOV_TOKEN }
185         directory: reports
186         fail_ci_if_error: false
187         verbose: true

```

Listing 4: Archivo completo `.github/workflows/ci-cd.yml`.

La confirmación de que Codecov efectivamente recibe los reportes no fue automática: la primera versión del job coverage (sin la línea `token`) marcaba el job en verde igualmente, porque

`fail_ci_if_error: false` evita que un fallo de subida rompa el pipeline; solo al expandir el log del paso “Upload coverage to Codecov” se encontró el error real, `{"message": "Token required - not valid tokenless upload"}`, oculto detrás de un *checkmark* verde. La causa fue doble: primero, la app de GitHub de Codecov solo estaba instalada sobre la cuenta personal de un integrante y no sobre la cuenta propietaria del repositorio, por lo que **TiendaTech** no aparecía en su panel; una vez el propietario del repositorio instaló la app y autorizó el acceso, siguió exigiéndose un `CODECOV_TOKEN` explícito (Codecov v5 ya no acepta subidas sin token para repositorios privados), obtenido desde *Settings* → *General* → *Repository Upload Token* en Codecov y agregado como secreto en *Settings* → *Secrets and variables* → *Actions* del repositorio. Con el secreto referenciado en el paso (línea `token: ${ secrets.CODECOV_TOKEN }` del Listado 4), el log del commit `dd72b3d4` confirma la subida real: `Your upload is now queued for processing`, con `status_code=200`, y el panel de Codecov para ese mismo commit reporta **75.93 %** de cobertura agregada sobre `main`.

8. Observabilidad

Este criterio (D6: instrumentación con OpenTelemetry, métricas Prometheus, registros estructurados con `trace_id` y dashboard operativo en Grafana con los seis paneles obligatorios) se declara **no implementado** en esta entrega. No hay carpeta `ops/` en el repositorio, ni `opentelemetry-spring-boot-starter`, ni `prometheus/grafana` en `docker-compose.yml`, ni logs en formato JSON con identificador de traza en la configuración de Logback de los servicios revisados. Esta decisión de alcance ya fue declarada en la Sección 1.1 y se retoma con su justificación completa en la Discusión (Sección 10): la instrumentación de las tres señales de observabilidad es, de todo lo pendiente en la rúbrica, la de mayor costo de infraestructura por punto de nota, y compite directamente por el mismo tiempo que se usó para asegurar la persistencia distribuida seleccionada como prioridad (Sección 4.4).

9. Evaluación ISO/IEC 25010

Este criterio (D7: medir cinco características del modelo ISO/IEC 25010:2023 con métricas objetivas, repeticiones y un intervalo de confianza del 95 %) se declara **parcialmente implementado** en esta entrega. Dos de las cinco características ya cuentan con evidencia automatizada y reproducible, generada por el pipeline descrito en la Sección 7.3: Mantenibilidad, en la métrica de cobertura de pruebas (≥ 70 % en los seis microservicios y en **armado-ia**, Tabla 3); y Seguridad, verificada por `GatewayIntegrationTest`, que confirma que todos los endpoints protegidos evaluados retornan 401 sin token. La segunda métrica de Mantenibilidad — complejidad ciclomática < 10 — no se ha medido con una herramienta estática dedicada (p. ej. PMD o Checkstyle) y queda pendiente. Las tres características restantes — Disponibilidad (uptime ≥ 99.5 % en ventana continua de 1 h), Rendimiento (latencia p95 < 500 ms con 50 usuarios) y Fiabilidad (tasa de errores 5xx < 1 %) — dependen del protocolo experimental con Locust y de la instrumentación de observabilidad (Sección 8), ninguna de

las dos implementada aún, por lo que se mantienen declaradas como **no implementado** en este corte. No existe `docs/experimentos/protocolo-e4.md` ni `docs/experimentos/resultados/iso25010.csv` en el repositorio. El equipo prefiere declarar este estado mixto con precisión — dos de cinco características con evidencia real, tres de cinco pendientes de un mismo bloqueador compartido — antes que reportar el criterio como enteramente cumplido o enteramente incumplido.

10. Discusión y amenazas a la validez

10.1. Interpretación de resultados y alcance no cubierto

La Entrega 3 cerró declarando como trabajo posterior explícito la calidad de código, la observabilidad y la evaluación experimental formal del sistema (Sección 1). Frente a esa línea base, esta entrega avanza de forma desigual entre esos tres frentes, y esa desigualdad es en sí misma el hallazgo principal de la Discusión.

El avance más sólido es arquitectónico: los seis microservicios Java completaron el refactor en capas con puertos y adaptadores (Sección 4), y la persistencia distribuida pasó de ser un perfil experimental paralelo en E3 a ser la base de datos real del sistema, ahora protegida con mTLS. Este resultado es consistente con la prioridad que el equipo declaró en `docs/plan-cierre-e4.md`: asegurar primero la infraestructura de la que todo lo demás depende.

El avance en calidad de código es parcial y concentrado: cuatro de los seis microservicios Java tienen pruebas unitarias reales, mientras que los dos que concentran la lógica de negocio más nueva de esta entrega (recepción de órdenes, costeo, auto-desactivación) no tienen ninguna (Sección 7.1). Esto invierte la prioridad que el propio equipo se había fijado por escrito — cubrir primero la lógica nueva antes que perseguir cobertura pareja — y es una inconsistencia entre lo planeado y lo ejecutado que este documento prefiere señalar explícitamente en vez de omitir. De forma similar, el pipeline de CI/CD cubre de forma parcial cuatro de los siete *jobs* esperados (Sección 7.3), y los contratos Pact no se iniciaron (Sección 7.2).

La evaluación experimental formal — el tercer frente declarado pendiente en E3 — sigue sin abordarse en E4: la observabilidad distribuida y la evaluación ISO/IEC 25010 (Secciones 8 y 9) se declaran explícitamente fuera de alcance, no por desconocimiento de las herramientas sino porque instrumentar las tres señales de observabilidad (métricas, registros, trazas) y luego ejecutar un protocolo experimental con repeticiones e intervalos de confianza sobre esas métricas es, de todo lo pendiente en la rúbrica, lo que exige más tiempo de infraestructura por punto de nota disponible. Priorizar la persistencia distribuida seleccionada como base de todo el sistema, en lugar de la observabilidad, fue una decisión consciente del equipo bajo la restricción de tiempo real que enfrentó esta entrega, y no un descuido.

La instrumentación de observabilidad distribuida (OpenTelemetry, Prometheus, Grafana) y la evaluación experimental ISO/IEC 25010 — que depende de tener esas métricas funcionando — quedaron fuera del alcance de esta entrega por restricción de tiempo del equipo, y se declaran

explícitamente como trabajo futuro en las Conclusiones (Sección 11). No se fabrican métricas ni dashboards simulados para cubrir estos puntos (Secciones 8 y 9).

10.2. Amenazas a la validez (D9.3)

Sobre las mediciones y verificaciones reportadas en la Sección 7, corresponde declarar tres amenazas a su validez siguiendo la clasificación estándar de estudios empíricos en ingeniería de software.

Validez de constructo: la cobertura de pruebas se reportó por *presencia de archivos de prueba por servicio*, no por un porcentaje de líneas o ramas cubiertas medido con una herramienta como JaCoCo. Solo `pedidos-service` tiene JaCoCo configurado en su `pom.xml`; en el resto del backend, "tiene pruebas" y "tiene buena cobertura" no son estrictamente lo mismo, y este documento no confunde ambas cosas al reportar el estado del servicio.

Validez interna: la única prueba de integración real contra una base de datos (`CockroachDbIntegrationTest` en `pedidos-service`) corre contra un contenedor CockroachDB de un solo nodo levantado por Testcontainers, mientras que el sistema en ejecución real usa un clúster de tres nodos con mTLS. El comportamiento verificado en la prueba automatizada no cubre escenarios de consenso Raft entre réplicas, conflictos de rango entre nodos, ni la ruta de conexión cifrada.

Validez externa: las pruebas manuales de extremo a extremo de esta entrega (recepción de órdenes, actualización de inventario, auto-desactivación de productos) se ejecutaron principalmente contra el entorno local descrito en la Sección 10.4, con datos de catálogo sintéticos y volumen bajo. No hay evidencia todavía de que el mismo flujo se comporte igual bajo carga concurrente o con el volumen de datos del clúster de AWS, precisamente el tipo de medición que correspondería a una evaluación ISO/IEC 25010 (Sección 9).

10.3. Reflexión ética (D9.2)

Se toma como marco de referencia el *ACM Code of Ethics and Professional Conduct* (2018), aplicado a tres decisiones de diseño concretas de **TiendaTech**, no como una discusión abstracta.

La primera es la protección de credenciales y datos personales (principio 1.6, *Respect privacy*; principio 2.9, *Design for security*). Las contraseñas se almacenan con `BCryptPasswordEncoder` de factor 12, nunca en texto plano; el token de sesión se guarda cifrado con Android Keystore en el cliente móvil y solo en `sessionStorage` (no en `localStorage` persistente) en el cliente web; y la conexión hacia la base de datos distribuida se cifra con autenticación mutua por certificado (mTLS) desde esta entrega, en lugar de la conexión sin cifrar heredada de E3.

La segunda es la decisión de auto-desactivar un producto cuando su costo recalculado

lado supera su precio de venta (documentada en la Sección 4 y en el código de `inventario-service/productos-service`). Esta es una decisión automatizada con impacto económico directo sobre el catálogo visible al cliente, y por tanto queda dentro del alcance del principio 2.5 (*Give comprehensive evaluation of computer systems... including analysis of possible risks*) y del principio 1.2 (*Avoid harm*): vender por debajo del costo perjudicaría al negocio, pero desactivar un producto sin aviso también tiene un costo (pérdida de visibilidad, posible error de cálculo transitorio). Por eso la implementación no es una decisión silenciosa e irreversible: queda registrada en el log del servicio (`LOG.warn` con el producto, el costo recalculado y el precio vigente), es visible y reversible por un administrador humano, y la reactivación exige explícitamente que el precio ya no esté por debajo del costo antes de permitirla — es decir, el sistema no permite reactivar "a ciegas" un producto que seguiría vendiéndose con pérdida.

La tercera es la honestidad en la comunicación de resultados, que el Code of Ethics exige en el principio 2.6 (*Perform work only in areas of competence*) y, sobre todo, en el compromiso general de integridad profesional: este mismo documento declara explícitamente, sección por sección, qué partes de la rúbrica se cumplen, cuáles se cumplen de forma parcial y cuáles no se abordaron (Observabilidad e ISO/IEC 25010 en las Secciones 8 y 9; ausencia de pruebas en dos microservicios en la Sección 7; ausencia de contratos Pact en la Sección 7.2), en vez de presentar el proyecto como completo cuando no lo está.

10.4. Reproducibilidad (D9.4)

El sistema completo puede levantarse con `dockercomposeup--build` contra el clúster CockroachDB de AWS usando las credenciales por defecto embebidas en `docker-compose.yml`, pero esto ata cualquier prueba local a un recurso compartido por todo el equipo y a una base de datos que no se puede resetear libremente. Para resolver esto, `docker-compose.override.yml` — que `docker compose` carga automáticamente junto al archivo principal, sin modificarlo — añade un nodo CockroachDB local de un solo contenedor (`crdb-local`, modo `-insecure`, sin necesidad de certificados) y un segundo contenedor de inicialización (`crdb-local-init`) que carga, en orden, el esquema (`schema.sql`), los datos semilla (`seeds.sql`) y dos catálogos de referencia críticos para que el sistema funcione de punta a punta: `product-management-reference.sql` (categorías, marcas, IVA) y `inventario-movimientos-reference.sql` (tipos y subtipos de movimiento de inventario, cuya ausencia fue precisamente la causa raíz de un defecto de esta misma entrega, donde las recepciones de órdenes de compra no actualizaban el stock ni en el entorno local ni en AWS).

Esta reproducibilidad tiene un límite honesto que se documenta aquí en vez de omitirse: los servicios Java no apuntan automáticamente a `crdb-local` por defecto, sino al clúster mTLS de AWS (Sección 4.4); para usar el entorno local hay que sobrescribir la variable de entorno `CRDB_DATASOURCE_URL` antes de levantar los contenedores (por ejemplo, `$env:CRDB_DATASOURCE_URL` en PowerShell), y esa variable es de alcance de sesión de terminal — si se abre una pestaña nueva sin volver a definirla, los contenedores reconstruidos en esa pestaña vuelven a apuntar por defecto a AWS mientras el resto del stack sigue apuntando

a `crdb-local`, produciendo un entorno mixto (*split-brain*) difícil de diagnosticar. El equipo confirmó este comportamiento durante las pruebas de esta entrega. La reproducibilidad "desde cero" en una máquina limpia es alcanzable pero no todavía a prueba de error humano; formalizar la variable en un archivo `.env` versionado (con placeholder) en vez de exportarla manualmente por sesión queda como mejora concreta pendiente.

11. Conclusiones y trabajo futuro

11.1. Cierre del ciclo E1–E4

La Tabla 4 resume el aporte específico de cada entrega, construida a partir de la lectura directa de `docs/entrega1/entrega1.pdf`, `docs/entrega2/entrega2.pdf`, `docs/entrega3/PFC3.tex` y la evidencia técnica en `docs/evidencias/`.

Cuadro 4: Trazabilidad de la evolución del proyecto entre E1 y E4.

Entrega	Enfoque	Aporte específico
E1 (27 may. 2026)	Propuesta y análisis	Especificación del problema (brecha de requerimientos formales de compatibilidad de hardware), delimitación explícita de la contribución al núcleo transaccional (autenticación, carrito, motor de compatibilidad, interfaz base del asistente inteligente) y roles del equipo (Arquitecto, Líder de Desarrollo, Responsable de Calidad, Responsable de Documentación).
E2 (6 jun. 2026)	Diseño arquitectónico	Diagramas C4 (contexto, contenedores, componentes), definición inicial de los servicios REST por dominio, contrato OpenAPI 3.0 y estrategia de contenerización con Docker para la arquitectura de microservicios propuesta.
E3 (28 jul. 2026)	Implementación y validación	Despliegue contenerizado real del frontend y seis microservicios sobre una base compartida, validado con 10 casos funcionales y 210 solicitudes (100 % de éxito, p95 <52ms). En paralelo, un perfil experimental desplegó un clúster CockroachDB de 3 nodos con fragmentación de <code>pedidos.orden</code> por trimestre (ADR-003) y factor de replicación 3 (ADR-004), verificado con pruebas de tolerancia a fallos documentadas en video (<code>docs/evidencias/tolerancia_fallos.mp4</code>), además de un pipeline analítico PySpark sobre 600 000 órdenes.
E4 (esta entrega)	Calidad, persistencia distribuida en producción y cierre	CockroachDB deja de ser un perfil experimental y pasa a ser la base de datos real de los seis microservicios en <code>docker-compose.yml</code> , ahora con conexión cifrada por certificado (mTLS); los seis servicios Java se refactorizaron a la arquitectura de puertos y adaptadores en cuatro capas, con 5 patrones GoF documentados (ADR-005); se corrigió el almacenamiento del token JWT en la web; se amplió — de forma desigual, ver Sección 7 — la cobertura de pruebas y el pipeline de CI/CD con publicación de imágenes Docker; y se declaran explícitamente fuera de alcance la observabilidad distribuida, la evaluación ISO/IEC 25010 y los contratos Pact.

Dos piezas de evidencia en `docs/evidencias/` documentan específicamente la transición entre el perfil experimental de E3 y el uso productivo de CockroachDB en E4: `integracion-pedidos-crdb.md` y `colision-serializable-controlada.md`, esta última relevante para el aislamiento `SERIALIZABLE` aplicado en `ordenes-proveedores-service` al enviar y cancelar órdenes de compra (Sección 4).

11.2. Conclusiones

Contrastado contra el alcance declarado en la Sección 1.1, el equipo cumplió de forma completa el refactor en capas y los patrones GoF (D1), avanzó de forma sólida en la persistencia distribuida segura (D4.2) y en la funcionalidad de ambas aplicaciones cliente (D2.1, D3.1), y

corrigió el manejo inseguro del token JWT identificado como pendiente antes de esta entrega. Cumplió de forma parcial la calidad de las aplicaciones cliente (D2.2, D3.2) y la pirámide de pruebas del backend (D5.1), con una brecha concreta y ya identificada: los dos microservicios con la lógica de negocio más nueva de la entrega no tienen pruebas propias. No abordó, y lo declara así en vez de simularlo, los contratos de consumidor Pact (D4.1), la observabilidad distribuida (D6) y la evaluación experimental ISO/IEC 25010 (D7).

El resultado neto es un sistema que evolucionó genuinamente en la dirección que E3 dejó pendiente — de *funciona* a *se puede sostener y verificar* — pero que no alcanzó a cubrir la totalidad de esa transición en el tiempo disponible. Esta entrega documenta esa transición de forma honesta antes que completa: cada sección declara explícitamente su estado real, sin inflar lo logrado ni ocultar lo pendiente, siguiendo el mismo estándar de honestidad de alcance aplicado en las entregas anteriores del proyecto.

11.3. Trabajo futuro

Se declara explícitamente como trabajo pendiente, en orden de prioridad según `docs/plan-cierre-e4.md`: cerrar el contrato Pact web↔backend (Sección 7.2); escribir pruebas unitarias para `ordenes-proveedores-service` y `ventas-service` sobre la lógica de recepción, costeo y auto-desactivación (Sección 7.1); instrumentar observabilidad distribuida y, a partir de ella, ejecutar la evaluación ISO/IEC 25010 (Secciones 8 y 9); ampliar el pipeline de CI/CD a los siete jobs completos y migrar la publicación de imágenes de Docker Hub a GHCR (Sección 7.3); implementar internacionalización en la web (Sección 5.1); y formalizar la variable `CRDB_DATASOURCE_URL` del entorno local reproducible en un archivo `.env` versionado (Sección 10.4).

Referencias

- [1] R. Taft, I. Sharif, A. Matei et al., «CockroachDB: The resilient geo-distributed SQL database,» en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*, Portland, OR, EE. UU., 2020, págs. 1493-1509. DOI: [10.1145/3318464.3386134](https://doi.org/10.1145/3318464.3386134).
- [2] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*, 4.^a ed. Cham, Suiza: Springer, 2020. DOI: [10.1007/978-3-030-26253-2](https://doi.org/10.1007/978-3-030-26253-2).
- [3] D. Kang, R. Jiang y S. Blanas, «Jigsaw: A Data Storage and Query Processing Engine for Irregular Table Partitioning,» en *Proceedings of the 2021 International Conference on Management of Data*, ép. SIGMOD '21, Association for Computing Machinery, 2021, págs. 898-911, ISBN: 978-1-4503-8343-1. DOI: [10.1145/3448016.3457547](https://doi.org/10.1145/3448016.3457547).
- [4] X. Zhou, G. Li, J. Feng, L. Liu y W. Guo, «Grep: A Graph Learning Based Database Partitioning System,» *Proceedings of the ACM on Management of Data*, vol. 1, n.º 1, 94:1-94:24, mayo de 2023. DOI: [10.1145/3588948](https://doi.org/10.1145/3588948).
- [5] P. J. Liu, C. P. Li y H. Chen, «Enhancing storage efficiency and performance: A survey of data partitioning techniques,» *Journal of Computer Science and Technology*, vol. 39, n.º 2, págs. 346-368, 2024. DOI: [10.1007/s11390-024-3538-1](https://doi.org/10.1007/s11390-024-3538-1).
- [6] B. M. Abdelhafiz, «Distributed Database Architecture,» en *2020 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)*, 2020. DOI: [10.1109/CSDE50874.2020.9411547](https://doi.org/10.1109/CSDE50874.2020.9411547).
- [7] M. Hasan y S. Yasmin, «Concurrency Control in Distributed Database System: Solution of the Anomalies & Challenges,» en *Proceedings of the 3rd International Conference on Computing Advancements (ICCA 2024)*, 2024. DOI: [10.1145/3723178.3723261](https://doi.org/10.1145/3723178.3723261).
- [8] G. Márquez, F. Osses y H. Astudillo, «Review of Architectural Patterns and Tactics for Microservices in Academic and Industrial Literature,» *IEEE Latin America Transactions*, vol. 16, n.º 9, págs. 2321-2327, 2018. DOI: [10.1109/TLA.2018.8789551](https://doi.org/10.1109/TLA.2018.8789551).
- [9] C. Zhong, S. Li, H. Huang et al., «Domain-Driven Design for Microservices: An Evidence-Based Investigation,» *IEEE Transactions on Software Engineering*, vol. 50, n.º 6, págs. 1425-1447, 2024. DOI: [10.1109/TSE.2024.3385835](https://doi.org/10.1109/TSE.2024.3385835).
- [10] P. Nawrocki, K. Wrona, M. Marczak y B. Sniezynski, «A Comparison of Native and Cross-Platform Frameworks for Mobile Applications,» *Computer*, vol. 54, n.º 3, págs. 18-27, 2021. DOI: [10.1109/MC.2020.2983893](https://doi.org/10.1109/MC.2020.2983893).
- [11] M. Shahin, M. A. Babar y L. Zhu, «Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices,» *IEEE Access*, vol. 5, págs. 3909-3943, 2017. DOI: [10.1109/ACCESS.2017.2685629](https://doi.org/10.1109/ACCESS.2017.2685629).
- [12] M. Hui, L. Wang, H. Li et al., «Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,» *Journal of Systems and Software*, vol. 220, págs. 112-232, 2025. DOI: [10.1016/j.jss.2024.112232](https://doi.org/10.1016/j.jss.2024.112232).

- [13] A. Decan, T. Mens, P. R. Mazrae y M. Golzadeh, «On the Use of GitHub Actions in Software Development Repositories,» en *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, 2022, págs. 235-245. DOI: [10.1109/ICSME55016.2022.00030](https://doi.org/10.1109/ICSME55016.2022.00030).
- [14] P. R. Mazrae, T. Mens, M. Golzadeh y A. Decan, «On the Usage, Co-Usage and Migration of CI/CD Tools: A Qualitative Analysis,» *Empirical Software Engineering*, vol. 28, n.º 2, pág. 52, 2023. DOI: [10.1007/s10664-022-10285-5](https://doi.org/10.1007/s10664-022-10285-5).
- [15] S. G. Saroar y M. Nayebe, «Developers’ Perception of GitHub Actions: A Survey Analysis,» en *27th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, ACM, 2023, págs. 121-130. DOI: [10.1145/3593434.3593463](https://doi.org/10.1145/3593434.3593463).
- [16] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018, Ch. 11, pp. 357–402, ISBN: 978-1-61729-454-9.
- [17] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994, ISBN: 978-0-201-63361-0.
- [18] S. Newman, *Building Microservices*, 2nd. Sebastopol, CA, USA: O’Reilly Media, 2021, Ch. 12–15.
- [19] T. Winters, T. Manshreck y H. Wright, *Software Engineering at Google: Lessons Learned from Programming Over Time*. Sebastopol, CA, USA: O’Reilly Media, 2020, ISBN: 978-1-4920-8278-1.
- [20] P. Valenzuela-Toledo, A. Bergel, T. Kehrer y O. Nierstrasz, «The Hidden Costs of Automation: An Empirical Study on GitHub Actions Workflow Maintenance,» *arXiv*, 2024. eprint: [2409.02366](https://arxiv.org/abs/2409.02366).
- [21] A. R. Faqih, A. Taufiqurrahman, J. H. Husen y M. K. Sabariah, «Empirical Analysis of CI/CD Tools Usage in GitHub Actions Workflows,» *Journal of Informatics and Web Engineering*, vol. 3, n.º 2, págs. 251-261, 2024. DOI: [10.33093/jiwe.2024.3.2.18](https://doi.org/10.33093/jiwe.2024.3.2.18).